

LifeTrack

Clinical Trial Management Platform
Complete Backend Architecture & Code Reference

Microservices · ASP.NET Core (.NET 10) · EF Core · JWT

A full line-by-line walkthrough of all 8 services

Gateway · Authentication · ProtocolSite · Patient

AdverseEvent · DocumentCompliance · AnalyticsReport · Notification

Plus: JWT lifecycle · Filters · Memory Caching · Shared.CL

Table of Contents

1. Overall Architecture & the Complete JWT Flow
2. Authentication.API — the JWT issuer (every layer, every file)
3. Gateway.API & Shared.CL — routing and cross-cutting concerns
4. ProtocolSite.API — protocols, sites, and business rules
5. AdverseEvent.API — adverse events and deviations
6. DocumentCompliance.API — regulatory documents
7. Patient.API — patients, enrollments, and visits
8. AnalyticsReport.API — KPI reports
9. Notification.API — the notification bell with row-level security
10. Practical Questions — ports, frontend auto-start, audit DbContext instances
11. Filters — what they are, where used, and how each works
12. Memory Caching — the complete version-bump concept
13. Complete Learning Summary

1. Overall Architecture & the Complete JWT Flow

How All 8 Services Work Together

The LifeTrack backend is a **microservices solution** built on ASP.NET Core (.NET 10). Each service is an independent web API with its own process, port, and database connection. They never call each other directly (except one saga). The only entry point for the Angular frontend is the **Gateway**.

The 8 services and their ports

Service	Port	Owns	Database
Gateway.API	5046	Route /gateway/* to the right service	none
Authentication.API	5033	Users, Roles, AuditLogs, JWT issuance	GovernanceDb
ProtocolSite.API	5193	Protocols, Sites, SiteProtocols	ClinicalDb
Patient.API	5095	Patients, Enrollments, Visits	ClinicalDb
AdverseEvent.API	5159	Adverse Events, Deviations	ClinicalDb
DocumentCompliance.API	5232	Documents, DocumentVersions	GovernanceDb
AnalyticsReport.API	5062	KPI Reports	ClinicalDb
Notification.API	5210	Notifications	GovernanceDb

Two databases satisfy the microservices requirement: **GovernanceDb** (auth/governance/documents/notifications) and **ClinicalDb** (clinical operational data — protocols, patients, adverse events, KPIs).

The request path, end to end

The flow of every request is: **Angular** → **Gateway (Ocelot)** → **downstream microservice** → **Shared.CL filters** → **controller** → **service** → **repository** → **DbContext** → **SQL Server**.

- Angular sends a request.** The interceptor (`auth.interceptor.ts`) automatically attaches `Authorization: Bearer <token>` to every outgoing HTTP call.
- Gateway receives it.** Ocelot's `ocelot.json` maps every `/gateway/<resource>` path to the right downstream port, for example `/gateway/adverse-events/5` becomes `http://localhost:5159/api/adverse-events/5`. Ocelot passes the `Authorization` header through unchanged. It does zero JWT validation itself.
- The downstream microservice receives the request.** Before the controller action runs, ASP.NET's filter pipeline fires `JwtAuthFilter` from `Shared.CL`. This filter reads `Jwt:Secret` from `appsettings.json`, calls `JwtSecurityTokenHandler.ValidateToken(...)`, and if valid writes the decoded `ClaimsPrincipal` into `HttpContext.User`. If invalid or expired, 401 is returned immediately.
- Role check.** If the action has `[RoleAuthorize(RolesEnum.Admin)]`, the `RoleAuthorizeFilter` checks `ClaimTypes.Role` from the now-populated `User` principal against the allowed roles. Wrong role gives 403.
- Controller runs.** The action can now safely read `User.FindFirstValue(ClaimTypes.NameIdentifier)` to get the current user's ID, or `ClaimTypes.Role` to branch logic by role.

The JWT Flow in Full Detail

Part A — Issuing the token (Login)

- `POST /gateway/auth/login` is routed by Ocelot to `Authentication.API:5033/api/auth/login`.
- `AuthController.Login()` calls `AuthService.LoginAsync(req)`.
- `AuthService` uses `IUserRepository` to look up the user by email.
- `BCryptPasswordHasher.Verify(plaintext, storedHash)` — BCrypt internally re-hashes the plaintext with the salt embedded in the stored hash and compares. If it fails, `AuthException("Invalid email or password")` gives a 401.
- If `user.IsActive == false`, `AuthException("Your login has been blocked")` gives a 401.
- `JwtTokenService.Generate(user)` builds a `List<Claim>`: - `JwtRegisteredClaimNames.Sub = user.UserID.ToString()` (the standard "subject") - `JwtRegisteredClaimNames.Email = user.Email` - `JwtRegisteredClaimNames.Jti = Guid.NewGuid()` (unique token id, prevents replay) - `ClaimTypes.NameIdentifier = user.UserID.ToString()` (duplicate of Sub, ASP.NET convention) -

`ClaimTypes.Name = user.Name - ClaimTypes.Role = role name string (e.g. "Investigator" )`

7. A `SymmetricSecurityKey` is built from `Encoding.UTF8.GetBytes(settings.Secret)`. The `SigningCredentials` specify `HmacSha256`.
8. A `JwtSecurityToken` is created with issuer, audience, claims, `notBefore =.UtcNow`, `expires =.UtcNow + 60min`.
9. `JwtSecurityTokenHandler.WriteToken(token)` serializes it to the three-part Base64url string: `header.payload.signature`.
10. `AuthResponse` is returned containing `Token`, `ExpiresAtUtc`, and a `SessionUser` (UserID, Name, Email, Role).
11. Angular stores the token in a service variable (not localStorage, not cookies) and saves `SessionUser` for role-based UI decisions.

Part B — Validating the token (every subsequent request)

Every service's `Program.cs` registers `JwtAuthFilter` globally via `o.Filters.Add<JwtAuthFilter>()`. The filter runs before every controller action across all services — they all share the same secret from their own `appsettings.json`.

Inside `JwtAuthFilter.OnAuthorizationAsync`:

1. Checks for `[AllowAnonymous]` on the endpoint metadata — if present, returns immediately (login, patient self-register are public).
2. Reads the `Authorization` header. If missing or doesn't start with `"Bearer "`, returns 401.
3. Extracts the raw token string.
4. Reads `Jwt:Secret` (falls back to `JWT_SECRET` environment variable for production).
5. Builds `TokenValidationParameters`: validates signature key, issuer, audience, lifetime, and allows 30 seconds of clock skew.
6. `tokenHandler.ValidateToken(token, params, out _)` — this one call does everything: Base64-decodes, verifies HMAC signature, checks `exp` and `nbf`, checks issuer/audience.
7. On success, sets `context.HttpContext.User = principal` (the decoded claims are now available everywhere).
8. On failure (expired, tampered, wrong secret), returns 401 with the exception message.

Part C — The special patient registration saga

`POST /gateway/auth/register/patient` runs a three-step saga:

1. Creates the `User` account in GovernanceDb (Auth.API).
2. Auto-logs in to get a JWT for that new user.
3. Uses that JWT to call `Patient.API:5095/api/patients` via `IHttpClientFactory`, creating the Patient record in ClinicalDb.

If step 3 fails, a **compensating action** calls `TryDeleteUserAsync` to roll back the User from step 1. This is a manual saga — there is no distributed transaction coordinator.

JWT token anatomy

- **Header:** `alg=HS256, typ=JWT`
- **Payload claims:** `sub` (UserID), `email`, `jti` (unique token id), `NameIdentifier`, `Name`, `Role`
- **Extra:** `iss=LifeTrack`, `aud=LifeTrackUsers`, `exp=now+60min`, `nbf=now`
- **Signature:** `HMAC-SHA256( Base64(header).Base64(payload), Jwt:Secret from appsettings )`

The Shared.CL Library — The Glue

Every microservice references the `Shared.CL` class library project. It provides:

Filters (registered in every service's `Program.cs`): `JwtAuthFilter` (validates token, populates `HttpContext.User`), `RoleAuthorizeFilter` / `RoleAuthorizeAttribute` (typed role enforcement), `DomainExceptionHandler` (`DomainException` → 400, `NotFoundException` → 404), and `ValidateModelFilter` (422 with field-level errors). `ActivityLogFilter` logs every successful request.

Middleware: `GlobalExceptionHandler` is the outermost safety net; it catches anything that escaped the filter pipeline and returns a structured `ErrorDetail` JSON body instead of a raw 500 HTML page.

DTOs: `ApiResponse<T>` (standard envelope), `PagedResponse<T>` (paged list), `ErrorDetail` (structured error body with `traceId`).

Enums: `RolesEnum` (typed enum: Admin=1, ClinicalTrialManager=2, Investigator=3, Patient=4, RegulatoryOfficer=5, DataManager=6, matching the seeded Roles table exactly).

Exceptions: `DomainException` (business rule violation) and `NotFoundException` (record not found).

2. Authentication.API — Complete Code Walkthrough

This is the most important service in the entire project. It owns the only JWT that every other service then validates.

Folder structure — Controllers (AuthController, UsersController, AuditLogsController, AuditEntriesController), Data (AuthDbContext, DbSeeder), DTOs (LoginRequest, RegisterRequest, PatientRegisterRequest, AuthResponse, SessionUser, UserResponse, UpdateUserRequest, ChangePasswordRequest, PagedResult), Enums (UserRole), Migrations, Models (User, Role, AuditLog, AuditEntry), Repositories (UserRepository, AuditLogRepository + interfaces), Services (AuthService, UserService, JwtTokenService, JwtSettings, BCryptPasswordHasher, AuditLogService + interfaces), and root config (Program.cs, appsettings.json). It references Shared.CL.

appsettings.json

```
"GovernanceDb": "Server=(localdb)\\MSSQLLocalDB;Database=governanceDb;..."
"Jwt": { "Secret": "ThisistheJWTKEY...", "Issuer": "LifeTrack",
        "Audience": "LifeTrackUsers", "ExpiryMinutes": "60" }
"ServiceUrls": { "PatientApi": "http://localhost:5095/" }
```

Three config sections: the connection string to `governanceDb` (used by `AuthDbContext`), the JWT settings (read into `JwtSettings` via `Configure<JwtSettings>`), and the Patient.API URL for the registration saga. The `Jwt:Secret` is the same string every other service also has in their own `appsettings.json` — that shared secret is what makes all eight services able to validate one single token without talking to Auth.API on each request.

Program.cs — the startup spine

```
builder.Services.AddControllers(o =>
{
    o.Filters.Add<JwtAuthFilter>(); // validates token on every request
    o.Filters.Add<ActivityLogFilter>(); // logs method+path+user after success
    o.Filters.Add<DomainExceptionHandler>(); // catches DomainException -> 400
    o.Filters.Add<ValidateModelFilter>(); // catches bad ModelState -> 422
})
.AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));
```

`AddControllers` registers the MVC pipeline. The four filters are added as **global** filters — they run for every single action in the entire service. `JsonStringEnumConverter` makes enums serialize as their string names (`"Investigator"`) instead of integers (`3`).

```
builder.Services.AddDbContext<AuthDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("GovernanceDb"));
```

Registers `AuthDbContext` with the DI container as a scoped service. EF Core creates one instance per HTTP request. `UseSqlServer` plugs in the SQL Server provider using the connection string from config.

```
var jwtSection = builder.Configuration.GetSection("Jwt");
builder.Services.Configure<JwtSettings>(jwtSection);
```

Reads the `Jwt` section from `appsettings.json` and binds it to the `JwtSettings` class. Any service that injects `IOptions<JwtSettings>` gets a populated object with `Secret`, `Issuer`, `Audience`, `ExpiryMinutes`.

```
builder.Services.AddHttpClient("PatientApi", client => {
    client.BaseAddress = new Uri("http://localhost:5095/");
    client.Timeout = TimeSpan.FromSeconds(10);
});
```

Registers a named `HttpClient` called `"PatientApi"`. This is used only by `AuthController.RegisterPatient` during the saga. Named clients in `IHttpClientFactory` are pooled and reused, avoiding the socket exhaustion problem that `new HttpClient()` causes.

```
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IPasswordHasher, BCryptPasswordHasher>();
builder.Services.AddScoped<IJwtTokenService, JwtTokenService>();
// ...
```

`AddScoped` means one instance per HTTP request. All service and repository registrations follow this pattern — the interface is the "key", the concrete class is the "value". Controllers only ever see the interface; they never instantiate the concrete class themselves.

```
app.UseGlobalExceptionHandler(); // must be FIRST -- catches everything below
app.UseCors("AngularDevClient");
```

`UseGlobalExceptionHandler` wraps the entire pipeline in a try/catch middleware. It has to be first so it can catch exceptions from every middleware and controller below it. CORS comes next — it must run before `MapControllers` to set the response headers before the route executes.

```
using (var scope = app.Services.CreateScope()) {
    var db = scope.ServiceProvider.GetRequiredService<AuthDbContext>();
    await db.Database.EnsureCreatedAsync();
    await DbSeeder.SeedAsync(db);
}
```

On every app start, this block creates the database if it doesn't exist, then calls the seeder. `EnsureCreatedAsync` is a shortcut — it creates tables by reading the model but doesn't run migrations. In production you'd use `MigrateAsync()` instead. `CreateScope()` is necessary because `AuthDbContext` is scoped and can't be resolved from the root container directly.

Models — the four entity classes

User.cs

```
[Table("Users")]
public class User {
    [Key] public long UserID { get; set; }
    [Required, MaxLength(200)] public string Name { get; set; }
    [Required] public int RoleID { get; set; }
    [Required, MaxLength(256)] public string Email { get; set; }
    [Required, MaxLength(256)] public string PasswordHash { get; set; }
    [MaxLength(32)] public string? Phone { get; set; }
    public bool IsActive { get; set; } = true;
    public Role? RoleNavigation { get; set; } // navigation property
    public ICollection<AuditLog> AuditLogs { get; set; } = [];
}
```

`[Table("Users")]` explicitly names the SQL table. `[Key]` marks the primary key — EF Core auto-generates it as `IDENTITY(1,1)`. `[Required]` adds a SQL `NOT NULL` constraint. `[MaxLength(256)]` sets `nvarchar(256)` instead of `nvarchar(max)`. `RoleNavigation` is a navigation property — EF Core populates it when you call `.Include(u => u.RoleNavigation)`. `IsActive = true` as a property initializer means new users start as active unless explicitly deactivated.

Role.cs

```
[Table("Roles")]
public class Role {
    [Key] public int RoleID { get; set; }
    [Required, MaxLength(100)] public string RoleName { get; set; }
    public ICollection<User> Users { get; set; } = [];
}
```

The `RoleID` is `int`, not `long`. In `OnModelCreating`, `.ValueGeneratedNever()` tells EF Core not to make it an auto-increment column — because the seeder inserts explicit values 1–6 and would conflict with identity generation.

AuditLog.cs — records explicit business events

```
[Table("AuditLogs")]
public class AuditLog {
    [Key] public long AuditID { get; set; }
    [Required] public long UserID { get; set; }
    [Required, MaxLength(500)] public string Action { get; set; }
    [Required] public DateTime ActionTime { get; set; } = DateTime.UtcNow;
    [ForeignKey(nameof(UserID))] public User User { get; set; } = null!;
}
```

This records things like `"USER_LOGIN"`, `"USER_REGISTERED"`, `"PASSWORD_CHANGED"`. It's written explicitly by `AuditLogService.RecordAsync()`. The `[ForeignKey(nameof(UserID))]` attribute explicitly tells EF Core which column to use for the navigation property.

AuditEntry.cs — records automatic EF Core change events

```
[Table("AuditEntries")]
public class AuditEntry {
    [Key] public long AuditEntryID { get; set; }
    public string EntityName { get; set; } // e.g. "User", "Role"
    public string? PrimaryKey { get; set; } // e.g. "42"
    public string Action { get; set; } // "Insert" | "Update" | "Delete"
    public long? ChangedByUserID { get; set; }
    public DateTime ChangedAt { get; set; }
}
```

This is different from `AuditLog`. It is written **automatically** by `AuthDbContext.SaveChangesAsync()` — you never write to it manually. Every time any entity is added/modified/deleted, a row is written here. `ChangedByUserID` is nullable because some operations happen during seed/startup before any user is logged in.

Data/AuthDbContext.cs — the context and its auto-audit trick

```
public DbSet<User> Users => Set<User>();
public DbSet<Role> Roles => Set<Role>();
public DbSet<AuditLog> AuditLogs => Set<AuditLog>();
public DbSet<AuditEntry> AuditEntries => Set<AuditEntry>();
```

The four `DbSets` are defined as expression-body properties using `Set<T>()` — functionally identical to `public DbSet<T> Name { get; set; }`.

```
private long? CurrentUserID {
    get {
        var raw = _httpCtx?.HttpContext?.User?.FindFirstValue(ClaimTypes.NameIdentifier);
        return long.TryParse(raw, out var id) ? id : null;
    }
}
```

This reads the currently logged-in user's ID from the JWT claim at the moment `SaveChangesAsync` is called. It uses `IHttpContextAccessor` (registered via `AddHttpContextAccessor()`). This is how `ChangedByUserID` gets populated automatically.

OnModelCreating — the fluent API configuration:

```
modelBuilder.Entity<Role>(e => {
    e.HasKey(r => r.RoleID);
    e.Property(r => r.RoleID).ValueGeneratedNever(); // no IDENTITY on Roles
    e.Property(r => r.RoleName).HasMaxLength(100).IsRequired();
});
modelBuilder.Entity<User>(e => {
    e.HasIndex(u => u.Email).IsUnique(); // unique email constraint at DB level
    e.Property(u => u.IsActive).HasDefaultValue(true);
    e.HasOne(u => u.RoleNavigation).WithMany(r => r.Users)
        .HasForeignKey(u => u.RoleID).OnDelete(DeleteBehavior.Restrict);
});
```

`.HasIndex(u => u.Email).IsUnique()` adds a `UNIQUE` constraint in SQL. `OnDelete(DeleteBehavior.Restrict)` means deleting a Role that still has users assigned throws a database error instead of cascading.

The auto-audit `SaveChangesAsync` override:

```

public new async Task<int> SaveChangesAsync() {
    var changedEntries = ChangeTracker.Entries()
        .Where(e => e.Entity is not AuditEntry && e.Entity is not AuditLog &&
            e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)
        .ToList();

    foreach (var entry in changedEntries) {
        var action = entry.State switch {
            EntityState.Added => "Insert",
            EntityState.Modified => "Update",
            EntityState.Deleted => "Delete",
            _ => "Unknown"
        };
        var pk = entry.Properties
            .FirstOrDefault(p => p.Metadata.IsPrimaryKey())?.CurrentValue?.ToString();

        AuditEntries.Add(new AuditEntry {
            EntityName = entry.Entity.GetType().Name,
            PrimaryKey = pk,
            Action = action,
            ChangedByUserID = CurrentUserID,
            ChangedAt = DateTime.UtcNow
        });
    }
    return await base.SaveChangesAsync();
}

```

Before every save, EF Core's `ChangeTracker` is inspected. Every entity that is being added/modified/deleted (except `AuditEntry` and `AuditLog` themselves, otherwise it would recurse forever) gets a matching `AuditEntry` row. Then `base.SaveChangesAsync()` saves all the original entities AND the new audit entries in one transaction. The `new` keyword is needed because we're hiding (not overriding) the base method.

Data/DbSeeder.cs

```

if (!await db.Roles.AnyAsync())
{
    db.Roles.AddRange(
        new Role { RoleID = 1, RoleName = "Admin" },
        // ... all 6 roles
    );
    await db.SaveChangesAsync();
}
if (!await db.Users.AnyAsync(u => u.Email == "admin@lifetrack.com"))
{
    db.Users.Add(new User {
        PasswordHash = BCrypt.Net.BCrypt.HashPassword("Admin@1234", workFactor: 12),
        IsActive = true
    });
    await db.SaveChangesAsync();
}

```

`AnyAsync()` is an idempotency guard — if the data already exists, don't insert it again. This makes the seeder safe to call on every startup. The `workFactor: 12` means BCrypt runs $2^{12} = 4096$ internal rounds — expensive to compute but that's intentional. Making the hash slow is how BCrypt defends against brute-force attacks.

Services/BCryptPasswordHasher.cs

```

public string Hash(string password) =>
    BCrypt.Net.BCrypt.HashPassword(password, WorkFactor);

public bool Verify(string password, string hash)
{
    try { return BCrypt.Net.BCrypt.Verify(password, hash); }
    catch (BCrypt.Net.SaltParseException) { return false; }
}

```

`Hash()` produces a string like `"$2a$12$..."` — the `$2a$` is the BCrypt version marker, `$12$` is the work factor, followed by a 22-char random salt and the 31-char digest. `Verify()` doesn't need the salt separately — it extracts it from the stored hash string. The `SaltParseException` catch turns a corrupted hash into a safe `false` return.

Services/JwtSettings.cs

```
public class JwtSettings {
    public string Secret { get; set; } = string.Empty;
    public string Issuer { get; set; } = string.Empty;
    public string Audience { get; set; } = string.Empty;
    public int ExpiryMinutes { get; set; } = 60;
}
```

A pure POCO — just a container that `Configure<JwtSettings>` populates from the config section. No logic.

Services/JwtTokenService.cs

```
public (string Token, DateTime ExpiresAtUtc) Generate(User user)
{
    var expires = DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes);

    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, user.UserID.ToString()), // standard subject
        new(JwtRegisteredClaimNames.Email, user.Email),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()), // unique token id
        new(ClaimTypes.NameIdentifier, user.UserID.ToString()), // ASP.NET convention
        new(ClaimTypes.Name, user.Name),
        new(ClaimTypes.Role, roleName)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Secret));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _settings.Issuer,
        audience: _settings.Audience,
        claims: claims,
        notBefore: DateTime.UtcNow,
        expires: expires,
        signingCredentials: creds
    );

    return (new JwtSecurityTokenHandler().WriteToken(token), expires);
}
```

`JwtRegisteredClaimNames.Sub` is the standard IANA claim name for subject. `ClaimTypes.NameIdentifier` is the ASP.NET-specific name for the same concept — both are included because different parts of the codebase use different conventions. The `Jti` (JWT ID) is a unique GUID per token. `SymmetricSecurityKey` takes the secret as a UTF-8 byte array. `HmacSha256` signs the header+payload to produce the signature. `WriteToken(token)` serializes to `header.payload.signature` in Base64url.

Services/AuthService.cs

RegisterAsync:

```
var email = req.Email.Trim().ToLowerInvariant();
if (await _users.EmailExistsAsync(email))
    throw new AuthException("A user with this email already exists.");

var user = new User {
    Name = req.Name.Trim(),
    Email = email,
    RoleID = req.RoleID,
    PasswordHash = _hasher.Hash(req.Password),
    IsActive = true
};
await _users.AddAsync(user);
await _audit.RecordAsync(user.UserID, "USER_REGISTERED");
```

The email is normalized to lowercase before both the check and the storage — this prevents `User@email.com` and `user@email.com` from being treated as different emails. The audit log entry is written after the user is saved (so `user.UserID` has been assigned by EF Core after the INSERT).

LoginAsync:

```
var user = await _users.GetByEmailAsync(email);
if (user is null || !_hasher.Verify(req.Password, user.PasswordHash))
    throw new AuthException("Invalid email or password.");
if (!user.IsActive)
    throw new AuthException("Your login has been blocked by the admin.");
```

The `user is null` check and the `!Verify` check are combined in one `if`. This is intentional — you get the same error message whether the email doesn't exist or the password is wrong. Separate messages would reveal which accounts exist (user enumeration attack). The `IsActive` check is separate and surfaces a different message.

ChangePasswordAsync:

```
if (!_hasher.Verify(req.CurrentPassword, user.PasswordHash))
    throw new AuthException("Current password is incorrect.");
user.PasswordHash = _hasher.Hash(req.NewPassword);
await _users.UpdateAsync(user);
```

You must provide your current password before setting a new one. The new password is re-hashed with a fresh BCrypt salt.

Services/UserService.cs — the cache versioning pattern

The problem it solves: `IMemoryCache.Remove()` with wildcard keys doesn't work. If you cache user lists under keys like `"users:list:page1:size10:search:"`, you can't do `_cache.Remove("users:list:*")`. So instead, `UserService` uses a **version counter**:

```
private int GetListVersion() =>
    _cache.GetOrCreate(ListVersionKey, e => {
        e.Priority = CacheItemPriority.NeverRemove;
        return 0;
    });

private void BumpListVersion() {
    var next = GetListVersion() + 1;
    _cache.Set(ListVersionKey, next, new MemoryCacheEntryOptions {
        Priority = CacheItemPriority.NeverRemove
    });
}
```

The version is a single integer stored at key `"users:list:version"`. Every list cache key embeds this version: `"users:list:v3:page1:size10:search:"`. When a user is created/updated/deleted, `BumpListVersion()` increments the counter to 4. All the old `v3` keys are now unreachable — they'll just expire naturally.

```
private void InvalidateUserCache(long userId) {
    _cache.Remove($"{ItemKeyPrefix}:{userId}"); // exact key -> remove immediately
    BumpListVersion();                          // list keys -> orphan via version bump
}
```

Individual user items use exact keys (`"users:item:42"`) so they can be removed directly. List cache uses the version trick.

Services/AuditLogService.cs

```
public Task RecordAsync(long userId, string action) {
    var log = new AuditLog { UserID = userId, Action = action, ActionTime = DateTime.UtcNow };
    return _repo.AddAsync(log);
}
```

Creates a new `AuditLog` row. Called after login, registration, password change, user update, user delete. Note this returns a `Task` (not `async Task`) — passing the task up without awaiting it in this method.

```
ActionTime = DateTime.SpecifyKind(l.ActionTime, DateTimeKind.Utc)
```

SQL Server stores `datetime2` without timezone info — it comes back with `DateTimeKind.Unspecified`. `SpecifyKind(Utc)` tells the JSON serializer to append `"Z"` to the ISO 8601 string, so Angular receives `"2025-05-25T10:02:08Z"` instead of `"2025-05-25T10:02:08"`. Without this the Angular date pipe would treat the time as local time.

Repositories/UserRepository.cs

```
public Task<User?> GetByIdAsync(long userId) =>
    _db.Users.Include(u => u.RoleNavigation).FirstOrDefaultAsync(u => u.UserID == userId);
```

`.Include(u => u.RoleNavigation)` is an **eager load** — EF Core generates a single SQL JOIN instead of a separate second query.

```
public async Task<(IEnumerable<User> Items, int TotalCount)> ListAsync(
    int page, int pageSize, string? search = null, bool? isActive = null)
{
    var query = _db.Users.Include(u => u.RoleNavigation).AsNoTracking();
    if (!string.IsNullOrEmpty(search)) {
        var s = search.Trim().ToLower();
        query = query.Where(u =>
            u.Name.ToLower().Contains(s) ||
            u.Email.ToLower().Contains(s) ||
            (u.RoleNavigation != null && u.RoleNavigation.RoleName.ToLower().Contains(s)));
    }
    if (isActive.HasValue)
        query = query.Where(u => u.IsActive == isActive.Value);
    query = query.OrderBy(u => u.Name);
    var total = await query.CountAsync();
    var items = await query.Skip((page - 1) * pageSize).Take(pageSize).ToListAsync();
    return (items, total);
}
```

`.AsNoTracking()` tells EF Core not to track these entities in the change tracker — since we're only reading, skipping tracking saves memory. `CountAsync()` runs `SELECT COUNT(*)` with all the same WHERE clauses before the pagination. The tuple return lets the service get both the page of data and the total count in one method call.

```
public async Task SoftDeleteAsync(long userId) {
    var u = await _db.Users.FindAsync(userId);
    if (u != null) { u.IsActive = false; await _db.SaveChangesAsync(); }
}
```

This is a **soft delete** — the row stays in the database with `IsActive = false`. The user can be reactivated later. `FindAsync` looks in the EF change tracker first, then hits the database only if not found.

Controllers/AuthController.cs

[AllowAnonymous] on Login and Patient registration:

```
[HttpPost("login")]
[AllowAnonymous]
public async Task<ActionResult<AuthResponse>> Login([FromBody] LoginRequest req)
```

`[AllowAnonymous]` bypasses the globally-registered `JwtAuthFilter`. Without this attribute, the filter would demand a valid token before the login endpoint itself — a chicken-and-egg problem.

The patient registration saga:

```
// Step 1: create User
createdUser = await _auth.RegisterAsync(registerReq);

// Step 2: auto-login to get JWT
authResponse = await _auth.LoginAsync(new LoginRequest { Email = req.Email, Password = req.Password });

// Step 3: call Patient.API with that JWT
var client = _httpFactory.CreateClient("PatientApi");
client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", authResponse.Token);
var response = await client.PostAsync("api/patients", content);

// Compensating action on any failure:
await TryDeleteUserAsync(createdUser.UserID);
```

Step 2 is necessary because Patient.API's `POST /api/patients` requires a valid JWT. A brand-new user doesn't have one yet, so the service auto-logs them in and uses that token. If Patient.API returns a non-success status or is unreachable, the compensating action rolls back the user account.

GetUserId() helper:

```
private long? GetUserId() {
    var r = User.FindFirstValue(ClaimTypes.NameIdentifier);
    return long.TryParse(r, out var id) ? id : null;
}
```

`User` here is `ControllerBase.User` — the `ClaimsPrincipal` that `JwtAuthFilter` wrote into `HttpContext.User`. `long.TryParse` safely converts it.

Controllers/UsersController.cs

```
[HttpDelete("{id:long}")]
[RoleAuthorize(RolesEnum.Admin)]
public async Task<IActionResult> Delete(long id)
{
    var actingId = GetUserId() ?? 0;
    if (actingId == id) return BadRequest(new { error = "Cannot delete your own account." });
    if (id == 1) return BadRequest(new { error = "The Super Admin account cannot be deleted." });
    ...
}
```

Two guard clauses before any service call. The first prevents an admin from soft-deleting their own account (which would lock them out). The second hard-codes protection for UserID 1 (the seeded Super Admin) — defense in depth.

Controllers/AuditEntriesController.cs

```
var items = await query
    .OrderByDescending(a => a.ChangedAt)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .Select(a => new AuditEntryDto { ... }) // projection
    .ToListAsync();
```

The `.Select(a => new AuditEntryDto {...})` is a **projection** — EF Core generates `SELECT AuditEntryID, EntityName, ...` rather than `SELECT *`. This controller injects `AuthDbContext` directly instead of a service or repository — acceptable for read-only query-heavy controllers with no business logic.

3. Gateway.API & Shared.CL — Complete Walkthrough

Gateway.API — the project is deliberately minimal

```
<!-- Gateway.API.csproj -->
<PackageReference Include="Ocelot" Version="24.1.0" />
```

The `.csproj` has only one real dependency: Ocelot. No EF Core, no JWT package, no controllers, no DbContext. The gateway is a pure proxy — it doesn't process data, doesn't validate tokens, doesn't touch the database. Its entire job is traffic routing.

Program.cs — the entire startup in ~20 lines

```
builder.Configuration.AddJsonFile("ocelot.json", optional: false, reloadOnChange: true);
builder.Services.AddOcelot(builder.Configuration);
```

Line 1 adds `ocelot.json` as a configuration source. `optional: false` means if the file is missing, the app crashes on startup with a helpful error. `reloadOnChange: true` means editing `ocelot.json` while running picks up new routes within seconds without a restart. Line 2 registers all of Ocelot's internal services (router, forwarder, middleware pipeline).

```
builder.Services.AddCors(options => {
    options.AddPolicy("AngularDevClient", policy =>
        policy.WithOrigins(
            "http://localhost:4200",
            "http://127.0.0.1:4200",
            "http://localhost:61229",
            "http://localhost:55276",
            "http://127.0.0.1:55276",
            "http://localhost:62536",
            "http://127.0.0.1:62536")
        .WithHeaders("Authorization", "Content-Type", "Accept", "X-Requested-With")
        .WithMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS")
        .AllowCredentials());
});
```

CORS must be configured on the Gateway because the browser sends requests to `localhost:5046` (the gateway), and the browser's same-origin policy blocks cross-origin calls. Multiple ports are listed because Angular's dev server can start on different ports depending on what's already in use.

```
app.UseCors("AngularDevClient");
await app.UseOcelot();
app.Run();
```

`UseCors` must come **before** `UseOcelot`. If reversed, CORS headers get added after Ocelot has already forwarded the request, so they never reach the browser. `await app.UseOcelot()` is the entire routing engine. There is no `app.MapControllers()` — the gateway has zero controllers.

ocelot.json — every route explained

Every route entry looks like this:

```
{
  "UpstreamPathTemplate": "/gateway/auth/{everything}",
  "UpstreamHttpMethod": ["Get", "Post", "Put", "Delete", "Patch", "Options"],
  "DownstreamPathTemplate": "/api/auth/{everything}",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [{ "Host": "localhost", "Port": 5033 }]
}
```

`UpstreamPathTemplate` is the URL pattern Angular calls — always starts with `/gateway/`. `{everything}` is a catch-all placeholder that matches any path segment after the prefix. So `/gateway/auth/login` becomes `/api/auth/login`. `DownstreamScheme: "http"` means Ocelot forwards over plain HTTP internally. `DownstreamHostAndPorts` is an array — Ocelot supports multiple entries for load balancing, but here each service has exactly one.

Why two entries per resource? Every resource has two routes: an exact path (`/gateway/patients`) for collection operations (GET list, POST create), and a wildcard (`/gateway/patients/{everything}`) for item operations (GET/PUT/DELETE by id). Ocelot matches routes in order — exact matches win over wildcards.

Route groups:

Upstream path	Downstream service
/gateway/auth/{everything}	Auth.API:5033 → /api/auth/{everything}
/gateway/users + /gateway/users/{everything}	Auth.API:5033 → /api/users
/gateway/audit-logs	Auth.API:5033 → /api/audit-logs
/gateway/governance-audit-entries	Auth.API:5033 → /api/audit-entries
/gateway/protocols/{everything}	ProtocolSite.API:5193
/gateway/sites/{everything}	ProtocolSite.API:5193
/gateway/site-protocols/{everything}	ProtocolSite.API:5193
/gateway/clinical-audit-entries	ProtocolSite.API:5193 → /api/audit-entries
/gateway/patients/{everything}	Patient.API:5095
/gateway/enrollments/{everything}	Patient.API:5095
/gateway/visits/{everything}	Patient.API:5095
/gateway/adverse-events/{everything}	AdverseEvent.API:5159
/gateway/deviations/{everything}	AdverseEvent.API:5159
/gateway/documents/{everything}	DocumentCompliance.API:5232
/gateway/kpi-reports/{everything}	AnalyticsReport.API:5062
/gateway/notifications/{everything}	Notification.API:5210

Note: both Auth.API and ProtocolSite.API expose `/api/audit-entries`, distinguished at the gateway by the `governance-` vs `clinical-` prefix.

GlobalConfiguration:

```
"GlobalConfiguration": { "BaseUrl": "http://localhost:5046" }
```

This tells Ocelot what its own public URL is — used when generating absolute URLs in responses (for example `Location` headers on 201 Created responses).

What Ocelot does NOT do here: no JWT validation, no rate limiting, no load balancing, no caching, no request aggregation. It is used purely as a reverse proxy. This keeps all security logic exactly once in `Shared.CL.JwtAuthFilter` inside each downstream service.

Shared.CL — the security and error brain

This is a `Microsoft.NET.Sdk` class library (not a web project) that references `Microsoft.AspNetCore.App` as a framework reference — giving it access to `IActionFilter`, `IMiddleware`, `ControllerBase`, `HttpContext` without being a web host itself.

```
<FrameworkReference Include="Microsoft.AspNetCore.App" />
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.0" />
```

Middleware/GlobalExceptionHandler.cs

```
public async Task InvokeAsync(HttpContext context)
{
    try { await _next(context); }
    catch (Exception ex) { await HandleExceptionAsync(context, ex); }
}
```

This is the ASP.NET Core middleware pattern. `_next` is a `RequestDelegate` — a function that calls the next middleware. By wrapping `_next(context)` in a try/catch, any exception from any part of the entire pipeline is caught here.

```
switch (ex)
{
    case DomainException:    statusCode = 400; errorTitle = "Business Rule Violation"; break;
    case NotFoundException:  statusCode = 404; errorTitle = "Not Found"; break;
    case UnauthorizedAccessException: statusCode = 401; break;
    default:                 statusCode = 500; errorTitle = "An unexpected error occurred"; break;
}
```

The critical security rule:

```
Details = statusCode < 500 ? [ex.Message] : [] // 500 errors get empty details
```

For 400/404 errors, the exception message is safe to surface. For 500 errors, the message might contain internal stack traces, SQL queries, or file paths — leaking those is a security risk. So the `details` array is left empty for 500s. `TraceId = context.TraceIdentifier` is a unique request ID that appears in both the response and server logs.

```
private static readonly JsonSerializerOptions _jsonOptions = new()
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
};
```

A `static readonly` field — created once and reused for every request. `JsonSerializerOptions` is expensive to construct, so making it static avoids allocating a new one per exception.

Filters/JwtAuthFilter.cs

```
public class JwtAuthFilter : IAsyncAuthorizationFilter
```

It implements `IAsyncAuthorizationFilter`, not `IAuthorizationFilter`. Authorization filters run before action filters and before model binding. So if the token is invalid, the request is rejected before even trying to bind the JSON body.

```
bool allowAnonymous = context.ActionDescriptor.EndpointMetadata
    .Any(em => em.GetType() == typeof(AllowAnonymousAttribute));
if (allowAnonymous) return;
```

`EndpointMetadata` contains all attributes applied to the action method and controller class. Checking for `AllowAnonymousAttribute` by its exact type is intentional — more precise than `is`.

```
string secretKey = Environment.GetEnvironmentVariable("JWT_SECRET")
?? _config["Jwt:Secret"]
?? throw new InvalidOperationException("Jwt:Secret is not configured.");
```

Production-first design: if the environment variable `JWT_SECRET` is set (as in a deployed container), it takes priority over `appsettings.json`. The development secret can stay in the config file without leaking into production.

```
var validationParameters = new TokenValidationParameters
{
    ValidateIssuerSigningKey = true,
    IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey)),
    ValidateIssuer = !string.IsNullOrEmpty(issuer), // conditional
    ValidIssuer = issuer,
    ValidateAudience = !string.IsNullOrEmpty(audience), // conditional
    ValidAudience = audience,
    ValidateLifetime = true,
    ClockSkew = TimeSpan.FromSeconds(30)
};
```

Issuer and audience validation are only activated if those values are configured. `ClockSkew = 30 seconds` means a token that expired 30 seconds ago is still accepted — accounts for slight clock differences between machines.

```
ClaimsPrincipal principal = tokenHandler.ValidateToken(token, validationParameters, out _);
context.HttpContext.User = principal;
```

`ValidateToken` does everything in one call: Base64url-decodes, recomputes the HMAC-SHA256 signature and compares, checks `exp`, `nbf`, `iss`, `aud`. Assigning to `context.HttpContext.User` makes the claims available everywhere downstream.

Filters/RoleAuthorizeFilter.cs + RoleAuthorizeAttribute.cs

```
public class RoleAuthorizeAttribute : TypeFilterAttribute
{
    public RoleAuthorizeAttribute(params RolesEnum[] roles)
        : base(typeof(RoleAuthorizeFilter))
    {
        Arguments = new object[] { roles };
    }
}
```

`TypeFilterAttribute` is a special ASP.NET Core attribute that creates a filter instance through DI. `Arguments` passes constructor arguments. So `[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager)]` creates a filter with `_allowedRoles = [Admin, ClinicalTrialManager]`.

```
string? userRole = context.HttpContext.User.FindFirstValue(ClaimTypes.Role);
bool hasRole = _allowedRoles.Any(r =>
    string.Equals(r.ToString(), userRole, StringComparison.OrdinalIgnoreCase));
```

`r.ToString()` on a `RolesEnum` gives the enum member name string. `ClaimTypes.Role` contains the same string packed into the token by `JwtTokenService.Generate()`. `OrdinalIgnoreCase` makes it robust to case mismatch.

Filters/ValidateModelFilter.cs

```
public void OnActionExecuting(ActionExecutingContext context)
{
    if (!context.ModelState.IsValid)
    {
        var errors = context.ModelState.Values
            .SelectMany(v => v.Errors)
            .Select(e => e.ErrorMessage)
            .ToList();

        context.Result = new UnprocessableEntityObjectResult(new ErrorDetail { ... });
    }
}
```

ASP.NET Core model binding runs **before** `OnActionExecuting` — by the time this filter runs, `[Required]`, `[MaxLength]`, `[EmailAddress]` have already been evaluated. Setting `context.Result` short-circuits the pipeline — the controller action never runs. `UnprocessableEntityObjectResult` returns HTTP 422.

Filters/ActivityLogFilter.cs

```
public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
{
    ActionExecutedContext executed = await next();

    if (executed.Exception is not null && !executed.ExceptionHandled) return;

    _logger.LogInformation(
        "Activity | {Method} {Path} | User: {UserId} ({Email}) | Status: {StatusCode}",
        method, path, userId ?? "anonymous", email ?? "-", statusCode);
}
```

`await next()` runs the entire action. The log statement runs **after** the action completes. The exception check skips logging when the action threw and the exception was not handled — those are logged by `GlobalExceptionHandler` instead, avoiding duplicates.

DTOs/ApiResponse<T>.cs

```
public static ApiResponse<T> Ok(T data, string? message = null) => new()
{
    Success = true, Data = data, Message = message
};

public static ApiResponse<T> Fail(string error) => new()
{
    Success = false, Errors = [error]
};
```

The factory pattern (`Ok` and `Fail` static methods) means controllers write `return Ok(ApiResponse<UserResponse>.Ok(result))`. The `init` accessor means properties can only be set during construction — the object is immutable after creation.

DTOs/ErrorDetail.cs

```
public sealed class ErrorDetail
{
    public int StatusCode { get; init; }
    public string Error { get; init; }
    public IReadOnlyList<string> Details { get; init; } = [];
    public string TraceId { get; init; }
    public DateTime TimestampUtc { get; init; } = DateTime.UtcNow;
}
```

`sealed` prevents inheritance. `IReadOnlyList<string>` means the caller cannot mutate the error list. `TraceId` is the request's correlation ID — searchable in logs.

Exceptions/DomainException.cs and NotFoundException.cs

```
public sealed class DomainException : Exception
{
    public DomainException(string message) : base(message) { }
}

public sealed class NotFoundException : Exception
{
    public NotFoundException(string resourceName, object key)
        : base($"{resourceName} with id '{key}' was not found.") { }
}
```

These are marker exception types — their value is in what they *are*, not what they do. By throwing `new NotFoundException("Patient", 42)`, the filter catches it specifically and returns 404 — without any try/catch in the controller.

Enums/RolesEnum.cs

```
public enum RolesEnum
{
    Admin = 1, ClinicalTrialManager = 2, Investigator = 3,
    Patient = 4, RegulatoryOfficer = 5, DataManager = 6
}
```

The explicit integer values are critical — they must match the `RoleID` values seeded by `DbSeeder.cs`. Using `[RoleAuthorize(RolesEnum.Admin)]` instead of `[Authorize(Roles = "Admin")]` means a typo like `"Adnim"` is a compile error, not a silent security bypass at runtime.

Extensions/ApplicationBuilderExtensions.cs

```
public static IApplicationBuilder UseGlobalExceptionHandler(this IApplicationBuilder app)
    => app.UseMiddleware<GlobalExceptionHandler>();
```

An extension method on `IApplicationBuilder` that wraps the verbose `app.UseMiddleware<...>()` call behind the clean `app.UseGlobalExceptionHandler()` name. Every service's `Program.cs` calls this as the first line of the middleware pipeline.

4. ProtocolSite.API — Complete Code Walkthrough

Port 5193 · Database: `clinicalDb` · Owns: Protocols, Sites, SiteProtocols

What this service is about

A **Protocol** is the research plan for a clinical trial — title, start/end date, lifecycle status (Draft → Active → Paused/Completed/Terminated). A **Site** is a hospital or clinic that runs the trial — name, location, status (Pending → Active → Suspended/Closed). A **SiteProtocol** is the link between them — it records which investigator is running a specific protocol at a specific site, the trial phase, and the initiation date. One protocol can run at many sites, one site can run many protocols.

appsettings.json

```
"ClinicalDb": "Server=(localdb)\\MSSQLLocalDB;Database=clinicalDb;..."
"Jwt": { "Secret": "ThisistheJWTKEY...", "Issuer": "LifeTrack", ... }
```

Two critical differences from Authentication.API: the connection string key is `"ClinicalDb"` pointing to a completely different SQL Server database. The `Jwt` section has the exact same secret as Auth.API — this is how the token issued by Auth.API is validated here without ever calling Auth.API.

Program.cs — startup differences from Auth.API

```
builder.Services.AddSwaggerGen(c => {
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "LifeTrack – Protocol & Site API", ... });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme { ... });
    c.AddSecurityRequirement(...);
});
```

This service includes full Swagger/OpenAPI documentation. `AddSecurityDefinition("Bearer")` makes the Swagger UI show a padlock button to paste a JWT for testing.

```
builder.Services.AddDbContext<ProtocolSiteDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("ClinicalDb"));
```

The connection string key `"ClinicalDb"` targets `clinicalDb`, not `governanceDb`.

```
// Run migrations manually: dotnet ef database update --project ProtocolSite.API
```

Unlike Auth.API which calls `EnsureCreatedAsync` on startup, ProtocolSite.API doesn't auto-create or migrate on startup. You run migrations manually. Also note `no app.UseCors()` — ProtocolSite.API has no CORS configuration because Angular never calls it directly. All requests go through the Gateway, which handles CORS.

Models — the three entity classes

Protocol.cs

```
[Table("Protocols")]
public class Protocol {
    [Key] public long ProtocolID { get; set; }
    [Required, MaxLength(300)] public string Title { get; set; }
    [Required] public DateTime StartDate { get; set; }
    public DateTime? EndDate { get; set; } // nullable: ongoing protocols have no end date
    [Required, MaxLength(50)] public string Status { get; set; } = "Draft";
    public ICollection<SiteProtocol> SiteProtocols { get; set; } = [];
}
```

`EndDate` is nullable — a protocol that is still ongoing has no end date. `Status = "Draft"` means a newly created protocol starts in draft. There is **no Phase property** on `Protocol` — the second migration (`MovePhaseToSiteProtocol`) moved it to `SiteProtocol`, recognizing that phase belongs to a specific site's execution of the protocol, not to the protocol itself.

Site.cs

```
[Table("Sites")]
public class Site {
    [Key] public long SiteID { get; set; }
    [Required, MaxLength(300)] public string Name { get; set; }
    [Required, MaxLength(500)] public string Location { get; set; }
    [Required, MaxLength(50)] public string Status { get; set; } = "Pending";
    public ICollection<SiteProtocol> SiteProtocols { get; set; } = [];
}
```

`Status = "Pending"` means a new site is not yet operational — it must be reviewed and set to Active before assignment.

SiteProtocol.cs — the most interesting model

```
[Table("SiteProtocols")]
public class SiteProtocol {
    [Key] public long SiteProtocolID { get; set; }
    [Required] public long SiteID { get; set; }
    [Required] public long ProtocolID { get; set; }
    public long? InvestigatorID { get; set; } // cross-service, nullable, no EF FK
    public DateTime? InitiationDate { get; set; }
    [Required, MaxLength(50)] public string Phase { get; set; } = "Preclinical";
    [Required, MaxLength(50)] public string Status { get; set; } = "Pending";
    [ForeignKey(nameof(SiteID))] public Site? Site { get; set; }
    [ForeignKey(nameof(ProtocolID))] public Protocol? Protocol { get; set; }
}
```

`InvestigatorID` is a **cross-service reference** — it points to a `UserID` in GovernanceDb (Auth.API's database), but there is no EF Core `ForeignKey` constraint and no navigation property to a `User` model. This is intentional in microservices architecture: ProtocolSite.API has no `User` entity, so EF Core can't enforce the FK. It is nullable because a site-protocol assignment can be created first and the investigator assigned later.

Data/ProtocolSiteDbContext.cs

The context follows the same auto-audit pattern as `AuthDbContext`. The `OnModelCreating`:

```
modelBuilder.Entity<SiteProtocol>(b => {
    b.HasOne(sp => sp.Site)
        .WithMany(s => s.SiteProtocols)
        .HasForeignKey(sp => sp.SiteID)
        .OnDelete(DeleteBehavior.Restrict); // can't delete Site with assignments
    b.HasOne(sp => sp.Protocol)
        .WithMany(p => p.SiteProtocols)
        .HasForeignKey(sp => sp.ProtocolID)
        .OnDelete(DeleteBehavior.Restrict); // can't delete Protocol with assignments
});
```

Both FK relationships use `DeleteBehavior.Restrict`. SQL Server throws a referential integrity error if you try to `DELETE` a `Site` or `Protocol` that still has `SiteProtocol` rows. It's a database-level guard.

```
b.HasIndex(sp => sp.SiteID);
b.HasIndex(sp => sp.ProtocolID);
b.HasIndex(sp => sp.InvestigatorID);
b.HasIndex(sp => sp.Phase);
b.HasIndex(sp => sp.Status);
```

Five indexes on `SiteProtocols` — the list endpoint filters by any of these. Without indexes, each filter would be a full table scan.

Migrations — the two-step schema history

`InitialCreate` creates four tables: `AuditEntries`, `Protocols`, `Sites`, `SiteProtocols`. The `SiteProtocols` table in this migration has no `Phase` yet.

`MovePhaseToSiteProtocol` — the `Up()` method:

```
migrationBuilder.DropIndex("IX_Protocols_Phase", "Protocols");
migrationBuilder.DropColumn("Phase", "Protocols");
migrationBuilder.AddColumn<string>("Phase", "SiteProtocols", ...);
migrationBuilder.CreateIndex("IX_SiteProtocols_Phase", "SiteProtocols", "Phase");
```

The `Phase` column was first placed on `Protocols` (a design mistake), then moved to `SiteProtocols` (the correct location). The

migration does this in one atomic transaction.

Repositories — the three repositories

ProtocolRepository.ListAsync:

```
if (!string.IsNullOrEmpty(status)) q = q.Where(p => p.Status == status);
if (!string.IsNullOrEmpty(search)) {
    var s = search.Trim().ToLower();
    q = q.Where(p => p.Title.ToLower().Contains(s));
}
var total = await q.CountAsync();
var items = await q.OrderByDescending(p => p.StartDate).Skip(...).Take(...).ToListAsync();
```

Status filter uses exact match (`==`). Search uses `.Contains()` which translates to SQL `LIKE '%s%'`. `OrderByDescending(p => p.StartDate)` means most recently started protocols come first.

ProtocolRepository.UpdateAsync:

```
public async Task UpdateAsync(Protocol protocol) {
    await _db.SaveChangesAsync(); // no _db.Update() call -- why?
}
```

This looks incomplete but is correct. The service calls `GetByIdAsync()` first — that loads the entity into the EF Core change tracker. The service mutates the properties directly. EF Core's change tracker detects those mutations and marks the entity as `Modified`. So calling `SaveChangesAsync()` alone is sufficient.

SiteProtocolRepository.ExistsAsync:

```
public Task<bool> ExistsAsync(long protocolId, long siteId, long investigatorId, long? excludeId = null) =>
    _db.SiteProtocols.AsNoTracking().AnyAsync(sp =>
        sp.ProtocolID == protocolId &&
        sp.SiteID == siteId &&
        sp.InvestigatorID == investigatorId &&
        (excludeId == null || sp.SiteProtocolID != excludeId.Value));
```

This checks for duplicate assignments. The `excludeId` parameter is used during **update** — when checking if the combination already exists, you don't want to match the record you're currently updating.

Services/ProtocolService.cs — business rules in depth

CreateAsync business rule:

```
if (req.EndDate.HasValue && req.EndDate.Value < req.StartDate)
    throw new DomainException("EndDate cannot be earlier than StartDate.");
```

`EndDate` is optional — only validate it if provided. `DomainException` is caught by `DomainExceptionFilter` and becomes HTTP 400.

UpdateAsync termination guard:

```
if (req.Status == "Terminated" && await _siteProtocols.HasProtocolAssignmentsAsync(id))
    throw new DomainException("Cannot terminate a protocol that has active site assignments...");
```

Before terminating, check if any `SiteProtocol` rows still reference it. This gives a human-readable error message instead of a raw SQL constraint violation.

DeleteAsync active guard:

```
if (protocol.Status == "Active")
    throw new DomainException("Cannot delete an active protocol. Pause or terminate it first.");
```

Even with no site assignments, you can't hard-delete an active protocol. Workflow must be Active → Paused/Terminated → delete.

Cache versioning — same pattern as Auth.API:

```
private int GetVersion() => _cache.GetOrCreate(VersionKey, e => {
    e.Priority = CacheItemPriority.NeverRemove; return 0;
});
private void BumpVersion() => _cache.Set(VersionKey, GetVersion() + 1,
    new MemoryCacheEntryOptions { Priority = CacheItemPriority.NeverRemove });
private void Invalidate(long id) { _cache.Remove($"{ItemPrefix}:{id}"); BumpVersion(); }
```

Each of the three services has its own version key (`protocols:list:version` , `sites:list:version` , `siteprotocols:list:version`) so invalidating one doesn't affect the others.

Services/SiteProtocolService.cs — the richest business logic

CreateAsync — five-step validation chain:

```
// Step 1: Site exists
var site = await _sites.GetByIdAsync(req.SiteID)
    ?? throw new DomainException($"Site {req.SiteID} not found.");

// Step 2: Site is Active
if (site.Status != "Active")
    throw new DomainException($"Site '{site.Name}' is currently {site.Status}. Only Active sites...");

// Step 3: Protocol exists
var protocol = await _protocols.GetByIdAsync(req.ProtocolID)
    ?? throw new DomainException($"Protocol {req.ProtocolID} not found.");

// Step 4: Protocol is Active
if (protocol.Status != "Active")
    throw new DomainException($"Protocol '{protocol.Title}' is currently {protocol.Status}...");

// Step 5: No duplicate
if (req.InvestigatorID.HasValue &&
    await _repo.ExistsAsync(req.ProtocolID, req.SiteID, req.InvestigatorID.Value))
    throw new DomainException("This investigator is already assigned...");
```

The `?? throw` pattern is C# null-coalescing with a throw expression — if `GetByIdAsync` returns null, the exception is thrown inline. Each step provides a specific, actionable error message.

ValidateInitiationDate — date window validation:

```
private static void ValidateInitiationDate(DateTime initiationDate, Protocol protocol)
{
    var date = initiationDate.Date; // strips time component

    if (date < protocol.StartDate.Date)
        throw new DomainException($"Initiation date must be on or after the protocol start date ({protocol.StartDate:MMM d, yyyy}).");

    if (protocol.EndDate.HasValue && date > protocol.EndDate.Value.Date)
        throw new DomainException($"Initiation date must be on or before the protocol end date ({protocol.EndDate.Value:MMM d, yyyy}).");
}
```

`.Date` strips the time component so you're comparing calendar dates only. The error message uses the `MMM d, yyyy` format (`Jun 15, 2025`).

Services/SystemService.cs — the two guarded operations

```
if (req.Status == "Closed" && await _siteProtocols.HasAssignmentsAsync(id))
    throw new DomainException("Cannot close a site that has assigned protocols...");
```

You can change status to anything else freely. But setting to `"Closed"` is blocked if any `SiteProtocol` rows reference this site. This only blocks `"Closed"` — you can still set a site to `"Suspended"` .

```
if (site.Status == "Active")
    throw new DomainException("Cannot delete an active site. Suspend or close it first.");
```

Controllers — the four controllers

```
[ApiController]
[Route("api/protocols")]
public class ProtocolsController : ControllerBase
{
    [HttpGet] // No [RoleAuthorize] -> all authenticated users can read
    [HttpGet("{id:long}")]
    [HttpPost] [RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager)]
    [HttpPut] [RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager)]
    [HttpDelete] [RoleAuthorize(RolesEnum.Admin)]
}
```

Read (GET) operations have no `[RoleAuthorize]` — any authenticated user can view. Write operations require `Admin` or `ClinicalTrialManager`. Delete requires `Admin` only.

SiteProtocolsController.Create returns CreatedAtAction:

```
return CreatedAtAction(nameof(Get), new { id = c.SiteProtocolID }, c);
```

This returns HTTP 201 instead of 200, with a `Location` header pointing to the new resource's URL.

Enums — three domain enums

```
public enum ProtocolStatus { Draft, Active, Paused, Completed, Terminated }
public enum SiteStatus     { Pending, Active, Suspended, Closed }
public enum TrialPhase      { Preclinical, Phase1, Phase2, Phase3, Phase4 }
```

These enums document the valid state machine values but are **not used as column types** in the model. The model stores `string Status` not `ProtocolStatus Status`. Why? EF Core stores enums as integers by default, and storing `"Active"` as `2` would make the database unreadable without the code. String storage makes the data self-describing.

5. AdverseEvent.API — Complete Code Walkthrough

Port 5159 · Database: `clinicalDb` · Owns: AdverseEvents, Deviations

What this service is about

An **Adverse Event (AE)** is any unexpected harmful reaction experienced by a patient during a clinical trial. It must be recorded immediately and tracked through a review-to-resolution lifecycle. A **Deviation** is a failure to follow the trial protocol as written at a specific site. Both are safety-critical records that regulatory officers review.

appsettings.json

```
"ClinicalDb": "Server=(localdb)\\MSSQLLocalDB;Database=clinicalDb;..."
"Jwt": { "Secret": "ThisistheJWTKEYformyprojectLifetrackINT26DfSA002", ... }
```

Same `clinicalDb` database as ProtocolSite.API. Same JWT secret as every other service. This service writes `AdverseEvents` and `Deviations` tables into the same physical database that ProtocolSite.API created.

Program.cs — what's different from ProtocolSite.API

```
builder.Services.AddScoped<IAdverseEventRepository, AdverseEventRepository>();
builder.Services.AddScoped<IDeviationRepository, DeviationRepository>();
builder.Services.AddScoped<IAdverseEventService, AdverseEventService>();
builder.Services.AddScoped<IDeviationService, DeviationService>();
```

Only four registrations — two repositories and two services. This service is deliberately leaner than ProtocolSite.API. No cross-service lookups, no saga, no complex validation chains. No `app.UseCors()` — same reason as ProtocolSite.API.

Models — the three model classes

AdverseEventRecord.cs

```
[Table("AdverseEvents")] // explicit table name -- avoids "AdverseEventRecords"
public class AdverseEventRecord
{
    [Key] public long EventID { get; set; }
    [Required] public long PatientID { get; set; } // cross-service: Patient.API
    [Required] public long ProtocolID { get; set; } // cross-service: ProtocolSite.API
    [Required, MaxLength(1000)] public string Description { get; set; } = string.Empty;
    [Required, MaxLength(50)] public string Severity { get; set; } = "Mild";
    [Required, MaxLength(50)] public string Status { get; set; } = "Reported";
    [Required] public DateTime ReportedDate { get; set; } = DateTime.UtcNow;
}
```

The class is named `AdverseEventRecord` (not `AdverseEvent`) to avoid namespace collision risk. `[Table("AdverseEvents")]` maps it to the correct SQL table name. Both `PatientID` and `ProtocolID` are **cross-service references** — raw `long` foreign keys with no EF navigation properties, because EF Core cannot enforce a SQL FK across services and this service has no `Patient` or `Protocol` entity classes.

Deviation.cs

```
[Table("Deviations")]
public class Deviation
{
    [Key] public long DeviationID { get; set; }
    /// <summary>FK to SiteProtocol in ProtocolSite DB -- cross-service, no EF nav property.</summary>
    [Required] public long SiteProtocolID { get; set; }
    [Required, MaxLength(1000)] public string Description { get; set; } = string.Empty;
    [Required, MaxLength(50)] public string Severity { get; set; } = "Minor";
    [Required, MaxLength(50)] public string Status { get; set; } = "Reported";
}
```

`SiteProtocolID` references the `SiteProtocols` table in `clinicalDb` (same database, but owned by ProtocolSite.API). Even though both tables are in the same database, there is no EF navigation property because AdverseEvent.API's `DbContext` doesn't

declare a `DbSet<SiteProtocol>`.

AuditEntry.cs

An identical copy of the `AuditEntry` class. It maps to the `AuditEntries` table which physically exists in `clinicalDb` — but that table was **created by ProtocolSite.API's migration**, not by this service.

Data/AdverseEventDbContext.cs — the `ExcludeFromMigrations()` trick

```
public DbSet<AdverseEventRecord> AdverseEvents => Set<AdverseEventRecord>();
public DbSet<Deviation> Deviations => Set<Deviation>();
public DbSet<AuditEntry> AuditEntries => Set<AuditEntry>();
```

Three `DbSet`s — but only two are created by this service's migrations.

```
modelBuilder.Entity<AuditEntry>()
    .ToTable("AuditEntries", t => t.ExcludeFromMigrations());
```

This is the single most important line in the `DbContext`. `ExcludeFromMigrations()` tells EF Core: "I know about this table and can write to it, but I am **not responsible for creating, altering, or dropping it**." Without this, running `dotnet ef migrations add` from `AdverseEvent.API` would generate a `CREATE TABLE AuditEntries` statement — and running that migration would crash because `ProtocolSite.API` already created the table. This is the correct pattern when multiple services share a table: exactly one service owns and migrates the table, all others use `ExcludeFromMigrations()`.

The `SaveChangesAsync` override is identical to `ProtocolSite.API`'s — inspects `ChangeTracker`, skips `AuditEntry` entities, writes an audit row for every changed entity. `OnModelCreating` configures five indexes on `AdverseEvents` (`PatientID`, `ProtocolID`, `Severity`, `Status`, `ReportedDate`) and two on `Deviations` (`SiteProtocolID`, `Status`).

DTOs — using record instead of class

```
public record CreateAdverseEventRequest(
    [Required] long PatientID,
    [Required] long ProtocolID,
    [Required, MaxLength(1000)] string Description,
    [Required, MaxLength(50)] string Severity,
    [Required] DateTime ReportedDate
);
```

This uses C# `record` with a **positional constructor** — concise syntax where constructor parameters and properties are declared in one line. Records are immutable by default. The validation attributes are applied to constructor parameters — ASP.NET Core's model binder reads them and populates `ModelState`. `CreateAdverseEventRequest` has **no `Status` field** — status is always set to `"Reported"` by the service. `UpdateAdverseEventRequest` does have `Status` because updates advance the lifecycle.

Repositories/AdverseEventRepository.cs

```
var q = _db.AdverseEvents.AsQueryable();
if (protocolId.HasValue) q = q.Where(x => x.ProtocolID == protocolId.Value);
if (patientId.HasValue) q = q.Where(x => x.PatientID == patientId.Value);
if (!string.IsNullOrEmpty(severity)) q = q.Where(x => x.Severity == severity);
if (!string.IsNullOrEmpty(status)) q = q.Where(x => x.Status == status);
q = q.OrderByDescending(x => x.ReportedDate);
var total = await q.CountAsync();
var items = await q.Skip((page - 1) * pageSize).Take(pageSize).ToListAsync();
```

Each `if` clause conditionally chains a `.Where()`. None of these run SQL individually — EF Core builds one composed `IQueryable`, and the SQL is only generated when `CountAsync()` and `ToToListAsync()` are called. This is the deferred execution model of LINQ.

```
public async Task UpdateAsync(AdverseEventRecord ae)
{
    _db.AdverseEvents.Update(ae); // explicit, unlike ProtocolSite's implicit approach
    await _db.SaveChangesAsync();
}
```

This uses `_db.Update(ae)` explicitly, which marks the entire entity as `Modified`. EF Core generates an UPDATE for all columns. Slightly less efficient than `ProtocolSite.API`'s implicit approach but simpler and correct.

Services/AdverseEventService.cs — three unique design decisions

Decision 1 — CreateAsync returns only the ID:

```
public async Task<long> CreateAsync(CreateAdverseEventRequest req)
{
    var ae = new AdverseEventRecord {
        PatientID      = req.PatientID,
        ProtocolID     = req.ProtocolID,
        Description    = req.Description,
        Severity       = req.Severity,
        Status         = "Reported",    // hardcoded, ignores any Status in req
        ReportedDate   = req.ReportedDate
    };
    var created = await _repo.AddAsync(ae);
    BumpVersion();
    return created.EventID;            // returns only the long ID
}
```

`Status = "Reported"` is always hardcoded — the service enforces the initial state regardless of what the caller sends. The service returns only the ID — the controller packages it as `{ eventID: 42 }` with a 201.

Decision 2 — UpdateAsync blocks "Closed" status:

```
if (req.Status == "Closed")
    throw new DomainException("'Closed' is not a valid adverse event status.");
```

The `AESStatus` enum has `Closed`, but the service throws `DomainException` if someone tries to set it — because in this clinical domain, AEs are closed by a regulatory review process outside this system. After updating all fields, `Invalidate(id)` clears the cache.

Decision 3 — ToResponse is a static private mapper:

```
private static AdverseEventResponse ToResponse(AdverseEventRecord x) =>
    new(x.EventID, x.PatientID, x.ProtocolID, x.Description, x.Severity, x.Status, x.ReportedDate);
```

Using the positional record constructor, one line maps the entity to the response DTO. It's `static` because it doesn't use instance fields — a pure transformation.

`DeviationService` mirrors this structure: hardcodes `Status = "Reported"` on create, blocks `"Closed"` on update, uses separate cache version keys.

Controllers — role separation in create vs update

```
[HttpPost]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
    RolesEnum.Investigator, RolesEnum.DataManager)]
public async Task<ActionResult> Create(...)

[HttpPut("{id:long}")]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
    RolesEnum.RegulatoryOfficer, RolesEnum.DataManager)]
public async Task<ActionResult> Update(...)
```

This split is clinically meaningful. An `Investigator` **reports** an adverse event (they're on the ground at the trial site and first see the problem) but cannot **update** it — updating involves changing the status through the review workflow, which belongs to `ClinicalTrialManager`, `RegulatoryOfficer`, or `DataManager`. So `Investigator` is in POST but not PUT. Conversely, `RegulatoryOfficer` is in PUT but not POST.

```
return StatusCode(201, new { eventID });
```

Uses `StatusCode(201, body)` instead of `CreatedAtAction(...)`. Simpler and still semantically correct. The same role separation applies in `DeviationsController`.

Enums — five enums, two slightly redundant

```
public enum AESeverity { Mild, Moderate, Severe, LifeThreatening, Fatal }
public enum Severity { Mild, Moderate, Severe, LifeThreatening, Fatal }
public enum AEStatus { Reported, UnderReview, Resolved, Closed }
public enum DeviationStatus { Reported, UnderReview, Resolved, Closed }
public enum DeviationType { Minor, Major, Critical }
```

`AESeverity` and `Severity` are duplicates — same values, different names, neither used in the model or service (the model stores `string Severity`). They serve as documentation. `DeviationType` was planned but not used. These are design notes for future cleanup.

Services/JwtSettings.cs — the intentional tombstone

```
// This file has been intentionally removed.
// JWT validation is now handled by Shared.CL.Filters.JwtAuthFilter, which reads
// Jwt:Secret directly from IConfiguration. No local JwtSettings class is needed.
```

This file exists as documentation — it tells any developer who opens the folder: "you're not missing a file, this is deliberate." Before `Shared.CL`, each service had its own `JwtSettings` class. After centralization, the local one was removed but the shell file was kept.

What makes AdverseEvent.API unique

No EF FK constraints at all — both `PatientID / ProtocolID` (on AE) and `SiteProtocolID` (on Deviation) are raw `long` columns with indexes but no SQL FOREIGN KEY constraints, because all referenced entities live in other services' domains. `ExcludeFromMigrations()` on `AuditEntries` — shared table pattern. `_db.Update(entity)` explicit vs change tracking. **Status hardcoded on create** — the caller cannot influence the initial status; it is always `"Reported"`.

6. DocumentCompliance.API — Complete Code Walkthrough

Port 5232 · Database: `governanceDb` · Owns: Documents

The immediate surprise — wrong database

```
"ConnectionStrings": { "GovernanceDb": "...Database=governanceDb;..." }
```

```
builder.Services.AddDbContext<DocumentDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("GovernanceDb"))
    .ConfigureWarnings(w => w.Ignore(...PendingModelChangesWarning)));
```

Every other clinical service uses `ClinicalDb`. DocumentCompliance.API uses `GovernanceDb` — the same database as Authentication.API. This is an architecture decision: regulatory documents are governance data, not clinical trial operational data. Documents like IRB approvals, consent forms, and regulatory submissions belong in the governance layer alongside users and roles.

The `.ConfigureWarnings(w => w.Ignore(PendingModelChangesWarning))` suppresses a warning EF Core emits when the current C# model doesn't match the snapshot — added during the migration bug fix.

Models/Document.cs — the one real entity

```
[Table("Documents")]
public class Document
{
    [Key] public long DocumentID { get; set; }
    [Required, MaxLength(300)] public string Title { get; set; } = string.Empty;
    // <summary>Document category -- maps to one of the 17 eTMF-aligned categories.</summary>
    [Required, MaxLength(100)] public string Category { get; set; } = "Other";
    public long? ProtocolID { get; set; } // nullable -- some docs not tied to any protocol
    [Required, MaxLength(20)] public string Version { get; set; } = "1.0";
    [Required] public long UploadedBy { get; set; } // cross-service: UserID
    [Required] public DateTime UploadedAt { get; set; } = DateTime.UtcNow;
    [Required, MaxLength(50)] public string Status { get; set; } = "Draft";
    [MaxLength(1000)] public string? Notes { get; set; }
    [MaxLength(2000)] public string? DocumentLink { get; set; } // Google Drive / OneDrive URL
}
```

`ProtocolID` is **nullable** — a document like a site master file may not be tied to any specific protocol. `UploadedBy` is a `long` referencing a `UserID` from `GovernanceDb` — since this service is in the same database as `Users`, you could add an EF FK, but it's kept as a raw cross-service reference at the code level. `DocumentLink` allows 2000 chars for a long signed URL. The "17 eTMF-aligned categories" refers to the electronic Trial Master File standard.

Models/DocumentVersion.cs — the unused planned entity

```
public class DocumentVersion
{
    public int DocumentVersionId { get; set; }
    public int DocumentId { get; set; }
    public Document Document { get; set; } = null!;
    public int VersionNumber { get; set; }
    public string FilePath { get; set; } = string.Empty;
    public long FileSizeBytes { get; set; }
    public string ContentType { get; set; } = string.Empty;
    public int UploadedByUserId { get; set; }
    public DateTime UploadedAt { get; set; }
    public string? Notes { get; set; }
}
```

This model exists in the `Models/` folder but has **no `DbSet`, no migration, and no service/repository**. It was planned for a full versioning system where each document could store multiple file versions. The current system stores only a `DocumentLink` URL and a simple `Version` string. This is a scaffold to complete later.

Data/DocumentDbContext.cs

```
public DbSet<Document> Documents => Set<Document>();
public DbSet<AuditEntry> AuditEntries => Set<AuditEntry>();
```

Only two `DbSet`s — `DocumentVersion` is intentionally absent. `OnModelCreating`:

```
modelBuilder.Entity<Document>(e => {
    e.ToTable("Documents");
    e.HasKey(d => d.DocumentID);
    e.Property(d => d.Title).HasMaxLength(300).IsRequired();
    e.Property(d => d.Category).HasMaxLength(100).IsRequired();
    e.Property(d => d.Version).HasMaxLength(20).IsRequired();
    e.Property(d => d.Status).HasMaxLength(50).IsRequired();
    e.Property(d => d.Notes).HasMaxLength(1000);
    e.Property(d => d.DocumentLink).HasMaxLength(2000);
    e.HasIndex(d => d.ProtocolID);
    e.HasIndex(d => d.Category);
    e.HasIndex(d => d.Status);
    e.HasIndex(d => d.UploadedBy);
    e.HasIndex(d => d.UploadedAt);
});
modelBuilder.Entity<AuditEntry>()
    .ToTable("AuditEntries", t => t.ExcludeFromMigrations());
```

Five indexes on `Documents`. `ExcludeFromMigrations()` on `AuditEntries` — same shared-table pattern (table owned by `Authentication.API` in `governanceDb`). The `SaveChangesAsync` override is identical to every other clinical service.

Migrations — two migrations and one important story

`InitialCreate` creates the `Documents` table:

```
migrationBuilder.CreateTable(
    name: "Documents",
    columns: table => new {
        DocumentID = table.Column<long>(...).Annotation("SqlServer:Identity", "1, 1"),
        ProtocolID = table.Column<long>(type: "bigint", nullable: false), // NOT NULLABLE here
        Type = table.Column<string>(type: "nvarchar(100)", ...), // "Type" not "Category"
        Version, UploadedBy, UploadedAt, Status
    }
);
```

Notice the `InitialCreate` migration has `ProtocolID` as `nullable: false` and the column is named `Type`. But the current `Document` model has `ProtocolID` as `long?` (nullable) and the property is named `Category`. This is evidence of the schema drift you fixed — the model changed (`Type`→`Category` rename, `ProtocolID` made nullable) but the migration wasn't regenerated cleanly. The snapshot also still shows `Type` and non-nullable `ProtocolID` — which is exactly why `PendingModelChangesWarning` is suppressed.

AddFileColumns — the raw SQL migration:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.Sql(@"
        IF NOT EXISTS (SELECT 1 FROM sys.columns
            WHERE object_id = OBJECT_ID('Documents') AND name = 'FileName')
            ALTER TABLE [Documents] ADD [FileName] nvarchar(500) NULL;

        IF NOT EXISTS (SELECT 1 FROM sys.columns
            WHERE object_id = OBJECT_ID('Documents') AND name = 'FilePath')
            ALTER TABLE [Documents] ADD [FilePath] nvarchar(1000) NULL;
    ");
}
```

This migration uses raw `migrationBuilder.Sql()` instead of `migrationBuilder.AddColumn<string>(...)`. Why raw SQL? Because `IF NOT EXISTS` idempotency guard is not available in EF Core's fluent API — `AddColumn` throws if the column already exists. The raw SQL checks `sys.columns` first, then adds only if missing. This is a defensive migration — safe to run multiple times. These two columns (`FileName`, `FilePath`) don't appear on the `Document` model class — added to support a file upload feature that wasn't fully implemented — another source of the warning.

DTOs/DocumentDtos.cs

```
public record CreateDocumentRequest(
    [Required, MaxLength(300)] string Title,
    [Required, MaxLength(100)] string Category,
    long? ProtocolID,
    [MaxLength(20)] string Version = "1.0",
    [MaxLength(1000)] string? Notes = null,
    long UploadedBy = 0,
    [MaxLength(2000)] string? DocumentLink = null
);
```

`ProtocolID` is `long?` matching the model. `Version = "1.0"` is a **default parameter value** in the positional record constructor. `UploadedBy = 0` is a placeholder — currently the controller passes `req.UploadedBy` from the request body, meaning Angular sends the user's ID explicitly. A more secure approach would read it from the token in the service layer. `UpdateDocumentRequest` adds `Status` as a required field.

Services/DocumentService.cs

CreateAsync — status hardcoded and link trimmed:

```
var doc = new Document {
    Title = req.Title,
    Category = req.Category,
    ProtocolID = req.ProtocolID,
    Version = req.Version,
    Notes = req.Notes,
    UploadedBy = req.UploadedBy,
    UploadedAt = DateTime.UtcNow, // server-side timestamp, not from caller
    Status = "Draft", // always Draft on create
    DocumentLink = string.IsNullOrEmpty(req.DocumentLink)
        ? null
        : req.DocumentLink.Trim() // sanitise: null if blank, trimmed if provided
};
```

`UploadedAt = DateTime.UtcNow` — the server stamps the upload time. `Status = "Draft"` — initial state enforced server-side. The `DocumentLink` sanitisation converts empty/whitespace strings to `null` and trims real URLs.

UpdateAsync — the approved-to-draft reversion rule:

```
if (doc.Status == "Approved" && req.Status == "Draft")
    throw new DomainException("An approved document cannot be reverted to Draft.");
```

This is the single business rule in this service. A document that has been officially `Approved` cannot be dragged back to `Draft`. The rule only blocks `Approved → Draft`. Other transitions like `Approved → Superseded` are allowed.

Repositories/DocumentRepository.cs

```
var q = _db.Documents.AsQueryable();
if (protocolId.HasValue) q = q.Where(d => d.ProtocolID == protocolId.Value);
if (!string.IsNullOrEmpty(status)) q = q.Where(d => d.Status == status);
if (!string.IsNullOrEmpty(category)) q = q.Where(d => d.Category == category);
q = q.OrderByDescending(d => d.UploadedAt);
```

Three optional filters. `OrderByDescending(d.UploadedAt)` means newest documents first — important for a compliance dashboard. The `UpdateAsync` uses `_db.Documents.Update(document)` explicitly.

Controllers/DocumentsController.cs

```
[HttpPost]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
    RolesEnum.RegulatoryOfficer, RolesEnum.DataManager)]
public async Task<ActionResult<DocumentResponse>> Create([FromBody] CreateDocumentRequest req)
{
    var created = await _svc.CreateAsync(req);
    return CreatedAtAction(nameof(Get), new { id = created.DocumentID }, created);
}
```

`Create` returns `CreatedAtAction` — HTTP 201 with `Location: /api/documents/42` header and the full `DocumentResponse` body. The most complete REST response of any create endpoint in the project. `Investigator` is **not** in the POST role list — investigators report adverse events but document management is for managers, regulatory officers, and data managers. `Update` returns the

full updated `DocumentResponse` (200 OK with body).

Enums

```
public enum DocumentStatus {  
    Draft, UnderReview, Approved, Rejected, Superseded  
}  
public enum DocumentCategory {  
    Protocol, ConsentForm, IRBApproval, SiteReport,  
    AdverseEventReport, RegulatorySubmission, Other  
}
```

`DocumentStatus` — five states. `Draft → UnderReview → Approved or Rejected`. An approved document replaced by a newer version goes to `Superseded`. `DocumentCategory` — `IRBApproval` is the Institutional Review Board approval required before any trial can start. `ConsentForm` is the patient informed consent document.

What makes DocumentCompliance.API unique

It uses `GovernanceDb` not `ClinicalDb` — a deliberate governance layer decision. **It has the most schema drift** — `InitialCreate` was created when the model was `Type + non-nullable ProtocolID`; the model then changed; `AddFileColumns` added DB columns not in the model; `PendingModelChangesWarning` is suppressed. The correct fix is to create a new migration capturing pending changes and sync the snapshot — a technical debt item. **It has a fully designed but unimplemented feature** — `DocumentVersion.cs` is parked and waiting.

7. Patient.API — Complete Code Walkthrough

Port 5095 · Database: `clinicalDb` · Owns: Patients, Enrollments, Visits

What this service is about

A **Patient** is a person enrolled in one or more clinical trials. A **PatientRecord** holds demographics — name, date of birth, contact info. An **Enrollment** links a patient to a specific `SiteProtocol` — the assignment of a patient to a trial at a particular site. A **Visit** is a single scheduled clinical appointment within that enrollment. The three entities form a chain: `Patient → Enrollment → Visit`.

Program.cs

Same `clinicalDb` connection string and same JWT secret. The `.csproj` includes `BCrypt.Net-Next` — a leftover reference (the service was built alongside Auth.API by Naveen) but BCrypt isn't actually used in Patient.API itself. Six `AddScoped` registrations — three repositories and three services.

Models — the three entities

PatientRecord.cs

```
[Table("Patients")]
public class PatientRecord
{
    [Key] public long PatientID { get; set; }
    public long? UserID { get; set; } // nullable cross-DB ref to Auth.API Users
    [Required, MaxLength(200)] public string Name { get; set; }
    [Required] public DateTime DOB { get; set; }
    [MaxLength(500)] public string ContactInfo { get; set; } = string.Empty;
    [Required, MaxLength(50)] public string EnrollmentStatus { get; set; } = "Screening";
    public ICollection<Enrollment> Enrollments { get; set; } = [];
}
```

`UserID` is nullable — a patient added by staff (investigator creates the record) has no `UserID`. A patient who self-registered through the Auth.API saga has a `UserID` linking them to their account in GovernanceDb. Since these are different databases, there is no EF FK — just a raw `long?`. Named `PatientRecord` not `Patient` to avoid namespace conflicts.

Enrollment.cs

```
[Table("Enrollments")]
public class Enrollment
{
    [Key] public long EnrollmentID { get; set; }
    [Required] public long PatientID { get; set; }
    [Required] public long SiteProtocolID { get; set; } // cross-service, no EF nav
    [Required] public DateTime EnrollmentDate { get; set; }
    public DateTime? ConsentDate { get; set; } // nullable -- consent may come later
    [Required, MaxLength(50)] public string Status { get; set; } = "Active";
    [MaxLength(500)] public string? WithdrawalReason { get; set; } // only on withdrawal
    [ForeignKey(nameof(PatientID))] public PatientRecord? Patient { get; set; }
    public ICollection<Visit> Visits { get; set; } = [];
}
```

`ConsentDate` is nullable because a patient can be enrolled before formal consent is recorded. `SiteProtocolID` is a cross-service reference to ProtocolSite.API. `WithdrawalReason` is only set when the status becomes `Withdrawn`. The `Patient` navigation property has an EF FK — both tables are in `clinicalDb`.

Visit.cs

```
[Table("Visits")]
public class Visit
{
    [Key] public long VisitID { get; set; }
    [Required] public long EnrollmentID { get; set; }
    [Required] public DateTime VisitDate { get; set; }
    [Required, MaxLength(50)] public string Status { get; set; } = "Scheduled";
    [MaxLength(1000)] public string Notes { get; set; } = string.Empty;
    [ForeignKey(nameof(EnrollmentID))] public Enrollment? Enrollment { get; set; }
}
```

`Notes` defaults to `string.Empty` not `null` — visit notes are considered always-present. `Status = "Scheduled"` means every new visit starts as scheduled.

Data/PatientDbContext.cs — two different delete behaviors

The most interesting design decision in this service — two EF relationships with different delete behaviors:

```
// Enrollment -> Patient: RESTRICT
e.HasOne(en => en.Patient).WithMany(p => p.Enrollments)
  .HasForeignKey(en => en.PatientID)
  .OnDelete(DeleteBehavior.Restrict);

// Visit -> Enrollment: CASCADE
e.HasOne(v => v.Enrollment).WithMany(en => en.Visits)
  .HasForeignKey(v => v.EnrollmentID)
  .OnDelete(DeleteBehavior.Cascade);
```

Restrict on Enrollment→Patient: You cannot delete a Patient who still has Enrollments. The patient's clinical record must be fully cleaned up first. **Cascade on Visit→Enrollment:** If an Enrollment is deleted, all its Visits are automatically deleted. A visit exists only within the context of an enrollment.

The unique filtered index on UserID:

```
e.HasIndex(p => p.UserID).IsUnique().HasFilter("[UserID] IS NOT NULL");
```

This creates a **UNIQUE** index but only for non-null values. One user account can only map to one patient record. But multiple patients can have `UserID = null` (staff-created records). The `HasFilter("[UserID] IS NOT NULL")` tells SQL to skip nulls from the uniqueness check. Without it, you could only have one null-UserID patient. `AuditEntries` uses `ExcludeFromMigrations()`.

Migrations/InitialCreate.cs — the cascade is in SQL

```
table.ForeignKey(
    name: "FK_Enrollments_Patients_PatientID",
    column: x => x.PatientID,
    principalTable: "Patients",
    principalColumn: "PatientID",
    onDelete: ReferentialAction.Restrict); // Restrict at DB level

table.ForeignKey(
    name: "FK_Visits_Enrollments_EnrollmentID",
    column: x => x.EnrollmentID,
    principalTable: "Enrollments",
    principalColumn: "EnrollmentID",
    onDelete: ReferentialAction.Cascade); // Cascade at DB level
```

Both delete behaviors are enforced at the SQL Server FK constraint level — not just in EF Core logic. Even raw SQL couldn't delete a Patient with Enrollments (Restrict), but deleting an Enrollment auto-deletes its Visits (Cascade).

Repositories/PatientRepository.cs — ExecuteUpdateAsync, a new EF pattern

```
public async Task UpdateEnrollmentStatusAsync(long patientId, string newStatus) =>
    await _db.Patients
        .Where(p => p.PatientID == patientId)
        .ExecuteUpdateAsync(s =>
            s.SetProperty(p => p.EnrollmentStatus, newStatus));
```

This is **EF Core's bulk update** — `ExecuteUpdateAsync`. It generates a single `UPDATE Patients SET EnrollmentStatus = @newStatus WHERE PatientID = @patientId`. No entity is loaded into memory, no change tracker is involved. Faster and safer than the

standard load → mutate → SaveChangesAsync. The interface comment explains: "bypasses the EF change-tracker entirely — safe to call even while other entities for the same patient are already tracked in the same DbContext."

Services/EnrollmentService.cs — the most complex logic in Patient.API

Duplicate enrollment check:

```
if (await _enrollments.ExistsAsync(req.PatientID, req.SiteProtocolID))
    throw new DomainException("This patient is already enrolled in the selected protocol and site.");
```

A patient can only be enrolled once per SiteProtocol.

SyncPatientStatusAsync — cross-entity status derivation:

```
private async Task SyncPatientStatusAsync(long patientId)
{
    var (allEnrollments, _) = await _enrollments.ListAsync(patientId, null, null, 1, 200);
    var derivedStatus = DerivePatientStatus(allEnrollments);
    await _patients.UpdateEnrollmentStatusAsync(patientId, derivedStatus);
    _cache.Remove($"patients:item:{patientId}");
    BumpVersion("patients:list:version");
}
```

This is called every time an enrollment is created or updated. It fetches all of a patient's enrollments, derives the correct `EnrollmentStatus` based on all of them, then updates the patient record. This keeps `Patients.EnrollmentStatus` in sync automatically.

```
private static string DerivePatientStatus(IEnumerable<Enrollment> enrollments)
{
    var statuses = enrollments.Select(e => e.Status)
                              .ToHashSet(StringComparer.OrdinalIgnoreCase);
    if (statuses.Count == 0) return "Screening";
    if (statuses.Contains("Active")) return "Active";
    if (statuses.Contains("Screening")) return "Screening";
    if (statuses.Contains("Completed")) return "Completed";
    return "Withdrawn";
}
```

Priority order: `Active > Screening > Completed > Withdrawn`. If a patient has one Active enrollment and one Withdrawn enrollment, the patient status is `Active` — they are still participating. Only when all enrollments are Withdrawn does the patient become Withdrawn. Note `SyncPatientStatusAsync` invalidates the `patients` cache from within the `enrollment` service — possible because both services share the same `IMemoryCache` singleton.

Withdrawal business rule:

```
if (enrollment.Status == "Withdrawn" && req.Status != "Withdrawn")
    throw new DomainException("A withdrawn enrollment cannot be reactivated.");
```

Once withdrawn, an enrollment is terminal. To re-enroll, a new Enrollment record must be created.

Services/VisitService.cs — enrollment state check before adding visits

```
public async Task<long> CreateAsync(CreateVisitRequest req)
{
    var enrollment = await _enrollments.GetByIdAsync(req.EnrollmentID);
    if (enrollment is null) throw new DomainException($"Enrollment {req.EnrollmentID} not found.");
    if (enrollment.Status == "Withdrawn")
        throw new DomainException("Cannot add visits to a withdrawn enrollment.");
    ...
}
```

You cannot schedule visits for a withdrawn enrollment. `UpdateVisitRequest` deliberately omits `EnrollmentID` — a visit's enrollment link cannot be changed after creation.

Controllers/PatientsController.cs — the IDOR protection pattern

```
[HttpPost]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
    RolesEnum.Investigator, RolesEnum.Patient)]
public async Task<ActionResult<PatientResponse>> Create([FromBody] CreatePatientRequest req)
{
    CreatePatientRequest finalReq = req;
    if (GetCallerRole() == nameof(RolesEnum.Patient))
    {
        var callerId = GetCallerId();
        finalReq = req with { UserID = callerId }; // record immutable override
    }
    var created = await _patients.CreateAsync(finalReq);
    return CreatedAtAction(nameof(Get), new { id = created.PatientID }, created);
}
```

This handles the Auth.API saga call. When `Patient` role calls this endpoint (during self-registration), the `UserID` in the request body is overridden with the caller's own JWT-derived ID. This prevents a malicious patient from submitting `{ UserID: 5 }` to create a record linked to someone else's account. The `req with { UserID = callerId }` syntax is a C# record **non-destructive mutation** — it creates a new record with only `UserID` changed.

EnrollmentsController — the `siteProtocolIds` comma-separated parsing:

```
IReadOnlyList<long>? spIds = null;
if (!string.IsNullOrEmpty(siteProtocolIds))
{
    spIds = siteProtocolIds.Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(s => long.TryParse(s.Trim(), out var id) ? id : 0L)
        .Where(id => id > 0)
        .ToList();
}
```

Angular sends `?siteProtocolIds=3,7,12` as a single query string parameter. This splits it, parses each to `long`, silently drops any that fail. The repository then generates SQL `WHERE SiteProtocolID IN (3, 7, 12)`.

8. AnalyticsReport.API — Complete Code Walkthrough

Port 5062 · Database: `clinicalDb` · Owns: KPIReports

What this service is about

This service stores pre-computed KPI (Key Performance Indicator) reports for the clinical trial dashboard. Reports are not generated by querying other services in real time — Angular's dashboard computes the metrics from data it has already fetched and sends the aggregated numbers here for storage and later retrieval. A `DataManager` creates the report, a `RegulatoryOfficer` reviews and marks it as `Reviewed`.

Models/KpiReport.cs — the fat single entity

```
[Table("KPIReports")]
public class KpiReport
{
    [Key] public long ReportID { get; set; }
    [Required, MaxLength(200)] public string Scope { get; set; } // e.g. "Protocol 5 Q1 2025"

    // Rates
    public double EnrollmentRate { get; set; } // % of target patients enrolled
    public double DropoutRate { get; set; } // % of enrolled who withdrew
    public double VisitComplianceRate { get; set; } // % of scheduled visits completed
    public double PatientRetentionRate { get; set; } // % still active at end of period

    // Totals
    public int TotalProtocols { get; set; }
    public int TotalPatients { get; set; }
    public int TotalEnrollments { get; set; }
    public int ActiveEnrollments { get; set; }
    public int TotalVisits { get; set; }

    // Adverse Events breakdown
    public int AECOUNT { get; set; }
    public int MildAEs { get; set; }
    public int ModerateAEs { get; set; }
    public int SevereAEs { get; set; }

    // Deviations breakdown
    public int DevCount { get; set; }
    public int ReportedDevs { get; set; }
    public int ResolvedDevs { get; set; }

    public DateTime? StartDate { get; set; }
    public DateTime? EndDate { get; set; }
    [Required] public DateTime GeneratedDate { get; set; } = DateTime.UtcNow;
    [MaxLength(20)] public string Status { get; set; } = "Draft";
    public DateTime? ReviewedAt { get; set; }
}
```

This is a reporting snapshot — all computed values stored as flat columns. `Scope` is a human-readable label like `"Global Q1 2025"`. `double` is used for rates (percentages like `73.5`). `int` for counts. `Status = "Draft"` until reviewed; `ReviewedAt` is set only on the review action.

Data/AnalyticsDbContext.cs — different CurrentUserID approach

Every other service reads `CurrentUserID` from `IHttpContextAccessor` injected into the constructor. `AnalyticsDbContext` takes a **different approach**:

```
// No IHttpContextAccessor injection
public long? CurrentUserID { get; set; } // public settable property instead
```

The `DbContext` doesn't inject `IHttpContextAccessor` at all. Instead `CurrentUserID` is a **public property** that the service sets manually before calling `SaveChangesAsync`. The comment says: "Set this before `SaveChangesAsync` to stamp the current user on audit rows." This is an alternative pattern — explicit assignment rather than automatic injection. Looking at `KpiReportService`, it doesn't actually set `CurrentUserID` — so audit entries from this service have `ChangedByUserID = null`. A minor oversight; the `IHttpContextAccessor` approach used by other services would have handled it automatically.

Migrations — four migrations tell a story

InitialCreate — creates `KPIReports` with only 6 columns: `ReportID`, `Scope`, `EnrollmentRate`, `DropoutRate`, `AECCount`, `GeneratedDate`. The service started minimal.

AddKpiFields — adds 13 more columns (`VisitComplianceRate`, `PatientRetentionRate`, `TotalProtocols`, `TotalPatients`, `TotalEnrollments`, `ActiveEnrollments`, `TotalVisits`, `MildAEs`, `ModerateAEs`, `SevereAEs`, `DevCount`, `ReportedDevs`, `ResolvedDevs`) using standard EF `AddColumn` calls with `defaultValue: 0`.

AddReviewStatus — adds `Status` and `ReviewedAt` using raw SQL with `IF NOT EXISTS` guards — same defensive pattern as `DocumentCompliance.API`:

```
migrationBuilder.Sql(@"
    IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('KPIReports') AND name = 'Status')
        ALTER TABLE [KPIReports] ADD [Status] nvarchar(20) NOT NULL DEFAULT 'Draft';
    IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('KPIReports') AND name = 'ReviewedAt')
        ALTER TABLE [KPIReports] ADD [ReviewedAt] datetime2 NULL;
");
```

AddStartDateAndReviewStatus — adds `StartDate`, `EndDate`, and re-guards `Status`/`ReviewedAt` with `IF NOT EXISTS`. The duplicate guard is safe because `IF NOT EXISTS` prevents errors. This migration history shows the KPI feature grew incrementally — basic rates, then breakdowns, then review workflow, then date range filtering.

Services/KpiReportService.cs — the ReviewAsync action

```
public async Task<KpiReportResponse?> ReviewAsync(long id)
{
    var report = await _repo.GetByIdAsync(id);
    if (report is null) return null;
    report.Status = "Reviewed";
    report.ReviewedAt = DateTime.UtcNow;
    await _repo.UpdateAsync(report);
    BumpVersion();
    return ToResponse(report);
}
```

The simplest possible state transition — load the report, set two fields, save. No business rule validation — any report can be reviewed. `BumpVersion()` is called but `Invalidate(id)` is not — meaning the item cache for this specific report may serve stale data for up to 5 minutes. A minor oversight; `Invalidate(id)` should also be called here.

KpiReportsController uses [HttpPatch] not [HttpPut]:

```
[HttpPatch("{id:long}/review")]
[RoleAuthorize(RolesEnum.RegulatoryOfficer)]
public async Task<ActionResult<KpiReportResponse>> Review(long id)
```

`PATCH` is semantically correct for a partial update — only `Status` and `ReviewedAt` change. The route `/api/kpi-reports/5/review` is a **sub-resource action route** — a REST pattern for operations more like actions than plain CRUD. No request body is needed because the service hardcodes `Status = "Reviewed"` and `ReviewedAt = UtcNow`. The create endpoint requires `Admin` or `DataManager`; only `RegulatoryOfficer` can review.

The repository also has a nice touch — `AddAsync` defaults the generated date if unset:

```
public async Task<KpiReport> AddAsync(KpiReport report)
{
    if (report.GeneratedDate == default) report.GeneratedDate = DateTime.UtcNow;
    _db.KpiReports.Add(report);
    await _db.SaveChangesAsync();
    return report;
}
```

9. Notification.API — Complete Code Walkthrough

Port 5210 · Database: `governanceDb` · Owns: Notifications

What this service is about

The notification bell. Every user sees their own notifications — new adverse events, enrollment updates, document review requests, visit reminders. Any role can create (send) a notification to a specific user by `UserID`. Notifications have two states: `Unread` and `Read`. Users can mark their notifications as read and delete them.

Program.cs

Uses `GovernanceDb` — same as `Auth.API` and `DocumentCompliance.API`. Notifications are user-centric governance data — they reference `UserID` from the `Users` table in `GovernanceDb`. Uses the same explicit-property `CurrentUserID` pattern as `AnalyticsDbContext` (no `AddHttpContextAccessor`).

Models/NotificationRecord.cs

```
[Table("Notifications")]
public class NotificationRecord
{
    [Key] public long NotificationID { get; set; }
    [Required] public long UserID { get; set; } // target user
    [Required, MaxLength(1000)] public string Message { get; set; }
    [Required, MaxLength(50)] public string Category { get; set; } = "Visit";
    [Required, MaxLength(20)] public string Status { get; set; } = "Unread";
    [Required] public DateTime CreatedDate { get; set; } = DateTime.UtcNow;
}
```

No navigation property to `User` — cross-service. `Category = "Visit"` as default but categories like `AdverseEvent`, `Enrollment`, `Document`, `Protocol` are used in practice. `Status` only has two states: `Unread` and `Read`.

Data/NotificationDbContext.cs — the composite index

```
e.HasIndex(n => new { n.UserID, n.Status }); // composite index
```

A **composite index** on two columns together. The query `GET /api/notifications/my?status=Unread` generates `WHERE UserID = 42 AND Status = 'Unread'`. With individual indexes, SQL Server would have to intersect two index scans. With a composite (`UserID`, `Status`) index, SQL Server does one seek and finds all matching rows directly — much faster for the most common query pattern (user loading their unread bell count). There are also individual indexes on `UserID`, `Status`, `Category`, and `CreatedDate`.

Services/NotificationService.cs — IDOR protection

IDOR stands for Insecure Direct Object Reference — a vulnerability where a user accesses or modifies another user's data by guessing or manipulating IDs.

```
public async Task<NotificationResponse?> MarkReadAsync(long id, long callerUserId)
{
    var n = await _repo.GetByIdAsync(id);
    if (n is null) return null;

    if (n.UserID != callerUserId)
        throw new UnauthorizedAccessException(
            "You can only mark your own notifications as read.");

    if (n.Status != "Read") // idempotency -- don't update if already read
    {
        n.Status = "Read";
        await _repo.UpdateAsync(n);
        Invalidate(id);
    }
    return ToResponse(n);
}
```

After fetching the notification by ID, the service checks `n.UserID != callerUserId`. If user 42 tries to mark notification 99 as read but it belongs to user 17, the service throws `UnauthorizedAccessException`. The controller catches this and returns `403 Forbid()`. Without this check, any authenticated user could mark any other user's notifications as read by guessing the ID. The `if (n.Status != "Read")` check is an **idempotency guard** — if already read, skip the database write. Same ownership check exists in `DeleteAsync`.

Controllers/NotificationsController.cs — three different access patterns

```
// 1. My notifications -- any authenticated user, filters by caller's UserID
[HttpGet("my")]
public async Task<ActionResult<PagedResult<NotificationResponse>>> ListMine(...)

// 2. All notifications -- Admin/CTM only
[HttpGet]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager)]
public async Task<ActionResult<PagedResult<NotificationResponse>>> ListAll(...)

// 3. Create -- all roles including Patient
[HttpPost]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager, RolesEnum.Investigator,
               RolesEnum.Patient, RolesEnum.RegulatoryOfficer, RolesEnum.DataManager)]
public async Task<ActionResult<NotificationResponse>> Create(...)
```

`GET /api/notifications/my` — no `[RoleAuthorize]`, but the service enforces that only the caller's own notifications are returned by filtering `WHERE UserID = callerUserId`. Role isn't relevant — your role doesn't determine whether you can see your own notifications. `GET /api/notifications` — only Admin and CTM can see all notifications across all users (admin oversight view). `POST /api/notifications` — all six roles can send notifications. The `UserID` in the request body is the **target** user, not the sender.

GetCurrentUserId reads from two possible claims:

```
var claim = User.FindFirstValue(ClaimTypes.NameIdentifier)
            ?? User.FindFirstValue("sub");
```

`ClaimTypes.NameIdentifier` is the ASP.NET Core convention. `"sub"` is the JWT standard claim name. Both are packed into the token. The double-check is a fallback in case one isn't present.

What makes Notification.API unique

Every other service exposes CRUD where the caller controls data through request bodies with authorization checks on who can write what. Notification.API adds something no other service has: **ownership-based access control at the record level**. `[RoleAuthorize]` alone can't protect this — it checks your role, not whether you own the specific record. The service explicitly loads the record, checks `n.UserID == callerUserId`, and throws `UnauthorizedAccessException` if ownership doesn't match. The controller translates that to `403 Forbid()`. This is the correct pattern for row-level security in a RESTful service.

Comparing the final three services

	Patient.API	AnalyticsReport.API	Notification.API
Entities	3 (Patient, Enrollment, Visit)	1 (KpiReport)	1 (Notification)
EF FK constraints	Patient→Enrollment (Restrict), Enrollment→Visit (Cascade)	None	None
Business logic	SyncPatientStatus, withdrawal terminal, visit-enrollment check	ReviewAsync sets Reviewed	IDOR ownership check
Unique pattern	cross-entity status derivation, ExecuteUpdateAsync, req with {}	PATCH sub-resource action, explicit CurrentUserID	record-level ownership security
Database	clinicalDb	clinicalDb	governanceDb
Migration count	1	4 (incremental growth)	1

10. Practical Questions Answered

Why the frontend starts automatically when you start the backend

The Gateway's `launchSettings.json` and the Visual Studio solution are configured to start multiple projects together. When you press Run, VS reads the solution's startup projects list and launches all of them — including the Angular frontend via a configured npm/node task. Check this at: right-click Solution → Properties → Startup Project → it's likely set to "Multiple startup projects."

Why port 62536 appeared

Look at the Gateway's CORS list:

```
"http://localhost:62536",  
"http://127.0.0.1:62536"
```

Angular's dev server (`ng serve`) tries port 4200 first. If something is already using 4200, it picks a random available port — 62536 was what the OS assigned that day. Angular reported it in the console, it got added to the CORS list, and now it's hardcoded there. The other ports in that list (55276, 61229) are previous random ports from earlier sessions where 4200 was taken.

The fix — in `angular.json`, pin the port so it always uses 4200:

```
"serve": {  
  "options": {  
    "port": 4200  
  }  
}
```

Then clean up all those random ports from the CORS list, keeping only `localhost:4200` and `127.0.0.1:4200`.

The audit injection: one big log file or separate, one object or many?

This question has two parts: the table and the object instances.

The AuditEntries table

There is **one single** `AuditEntries` table in each database. All clinical services (ProtocolSite, AdverseEvent, AnalyticsReport, Patient) write to the same `AuditEntries` table in `clinicalDb`. It is not one table per service. Rows pile into the same table and you distinguish them by `EntityName`:

AuditEntryID	EntityName	Action	ChangedByUserID	ChangedAt
1	Protocol	Insert	5	2025-06-01
2	AdverseEventRecord	Insert	3	2025-06-01
3	Document	Update	5	2025-06-01

So if you query `AuditEntries` you'll see rows from all services mixed together, identified by what entity was changed. (Note: Auth.API, DocumentCompliance.API, and Notification.API write to the `AuditEntries` table in `governanceDb` — a separate physical table but the same concept.)

The DbContext object — how many instances?

Every service registers its DbContext as `AddScoped` — which means **one DbContext instance per HTTP request**. If 10 requests hit 3 different services simultaneously:

```
Request A -> Patient.API          -> PatientDbContext instance #1 (lives, then dies)  
Request B -> AdverseEvent.API     -> AdverseEventDbContext instance #2 (lives, then dies)  
Request C -> Patient.API again    -> PatientDbContext instance #3 (lives, then dies)
```

Each request gets its own fresh DbContext — completely isolated. They don't share state, don't share the change tracker, don't interfere with each other. When the request ends, the DbContext is disposed. The `IHttpContextAccessor` injected into each

DbContext (to read `CurrentUserID`) is also scoped to the request, so each DbContext reads from its own request's JWT claims. There is no mixing of user IDs across requests.

Question	Answer
How many <code>AuditEntries</code> tables?	One per database (one in clinicalDb, one in governanceDb)
How many DbContext objects at runtime?	One per HTTP request per service
Do they share state?	No — fully isolated by Scoped lifetime
Can two requests corrupt each other's audit trail?	No — each has its own instance

The only "single big file" aspect is the database table itself — all audit rows go to one place per database. But the objects that write to it are ephemeral, per-request, and independent.

11. Filters — Complete Explanation

What is a Filter?

In ASP.NET Core, a **filter** is code that runs automatically at specific points in the request pipeline — before or after a controller action — without you writing that logic inside every action method. Think of it as a checkpoint that every request passes through.

The project has two kinds: **Global filters** registered in `Program.cs` via `o.Filters.Add<FilterName>()` that run on **every single action** across the entire service automatically, and **attribute filters** applied with `[RoleAuthorize(...)]` on specific actions or controllers only.

The Four Global Filters — registered in every service's Program.cs

```
builder.Services.AddControllers(o =>
{
    o.Filters.Add<JwtAuthFilter>();
    o.Filters.Add<ActivityLogFilter>();
    o.Filters.Add<DomainExceptionFilter>();
    o.Filters.Add<ValidateModelFilter>();
});
```

This exact block appears in **all 6 services** — Authentication.API, ProtocolSite.API, AdverseEvent.API, DocumentCompliance.API, AnalyticsReport.API, Notification.API. They all run the same four filters from `Shared.CL`.

Filter 1 — JwtAuthFilter (IAsyncAuthorizationFilter)

Where: Every service, every request. **What it does:** Runs before the controller action. Reads the `Authorization: Bearer <token>` header. Validates the JWT signature, expiry, issuer, and audience. If valid, writes the decoded claims into `HttpContext.User`. If invalid or missing, short-circuits the pipeline and returns 401 — the controller never runs.

```
// Step 1 -- skip if [AllowAnonymous] is on the action
bool allowAnonymous = context.ActionDescriptor.EndpointMetadata
    .Any(em => em.GetType() == typeof(AllowAnonymousAttribute));
if (allowAnonymous) return;

// Step 2 -- check header exists
if (!context.HttpContext.Request.Headers.TryGetValue("Authorization", out var authHeader)
    || !authHeader.ToString().StartsWith("Bearer "))
{
    context.Result = new UnauthorizedResult();
    return;
}

// Step 3 -- validate token
ClaimsPrincipal principal = tokenHandler.ValidateToken(token, validationParameters, out _);
context.HttpContext.User = principal; // now all claims available to the action
```

Real example: When an investigator hits `POST /gateway/adverse-events`, the Ocelot gateway forwards to AdverseEvent.API. `JwtAuthFilter` runs first, validates the token, sets `HttpContext.User`. Then `RoleAuthorizeFilter` checks the role. Then the controller runs. If the token was expired, 401 is returned and `AdverseEventsController.Create()` is never called. Only two actions have `[AllowAnonymous]`: `AuthController.Login()` and `AuthController.RegisterPatient()`.

Filter 2 — DomainExceptionFilter (IExceptionHandler)

Where: Every service, every action. **What it does:** Catches `DomainException` and `NotFoundException` thrown from the service layer and converts them into proper HTTP responses.

```

public void OnException(ExceptionContext context)
{
    if (context.Exception is DomainException domEx)
    {
        context.Result = new BadRequestObjectResult(new ErrorDetail
        {
            StatusCode = 400,
            Error       = domEx.Message
        });
        context.ExceptionHandled = true; // stops it going to GlobalExceptionHandler
    }
    else if (context.Exception is NotFoundException notFoundEx)
    {
        context.Result = new NotFoundObjectResult(new ErrorDetail
        {
            StatusCode = 404,
            Error       = notFoundEx.Message
        });
        context.ExceptionHandled = true;
    }
}

```

Real example: In `DocumentService.UpdateAsync`, `throw new DomainException("An approved document cannot be reverted to Draft.")` travels up to this filter, which returns `400 { error: "..." }`. The controller doesn't need any try/catch. Same in `AdverseEventService` (`"Closed" is not a valid adverse event status`), and `SiteProtocolService`'s five-step validation chain.

Filter 3 — ValidateModelFilter (IActionFilter)

Where: Every service, every POST and PUT action. **What it does:** Checks `ModelState.IsValid` before the action runs. If any `[Required]`, `[MaxLength]`, or `[EmailAddress]` annotation is violated, returns 422 with a list of field errors. The controller never runs.

```

public void OnActionExecuting(ActionExecutingContext context)
{
    if (!context.ModelState.IsValid)
    {
        var errors = context.ModelState.Values
            .SelectMany(v => v.Errors)
            .Select(e => e.ErrorMessage)
            .ToList();

        context.Result = new UnprocessableEntityObjectResult(new ErrorDetail
        {
            StatusCode = 422,
            Error       = "Validation failed.",
            Details     = errors
        });
    }
}

```

Real example: `CreateAdverseEventRequest` has `[Required] long PatientID`. If Angular sends a POST with no `PatientID`, this filter returns `{ "statusCode": 422, "error": "Validation failed.", "details": ["The PatientID field is required."] }`.

Filter 4 — ActivityLogFilter (IAsyncActionFilter)

Where: Every service, every action — but runs **after** the action completes. **What it does:** Logs method, path, user ID, email, and status code for every successful request.

```

public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
{
    ActionExecutedContext executed = await next(); // action runs here

    if (executed.Exception is not null && !executed.ExceptionHandled) return;

    _logger.LogInformation(
        "Activity | {Method} {Path} | User: {UserId} ({Email}) | Status: {StatusCode}",
        method, path, userId ?? "anonymous", email ?? "-", statusCode);
}

```

Real example: An admin updates a protocol → `PUT /api/protocols/5` → action runs → 200 returned → logs: `Activity | PUT /api/protocols/5 | User: 1 (admin@lifetrack.com) | Status: 200`. This creates a runtime activity trail in server logs — separate from the database `AuditEntries`.

The One Attribute Filter — RoleAuthorizeFilter

Where: Specific actions only, in every clinical service. Applied selectively per action using `[RoleAuthorize(...)]`.

```
[HttpPost]
[RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager, RolesEnum.Investigator)]
public async Task<ActionResult> Create(...)
```

What it does: After `JwtAuthFilter` has validated the token and set `HttpContext.User`, this filter reads `ClaimTypes.Role` from those claims and checks if it's in the allowed list. If not, 403 Forbidden.

```
string? userRole = context.HttpContext.User.FindFirstValue(ClaimTypes.Role);
bool hasRole = _allowedRoles.Any(r =>
    string.Equals(r.ToString(), userRole, StringComparison.OrdinalIgnoreCase));

if (!hasRole)
{
    context.Result = new ObjectResult(new ErrorDetail { StatusCode = 403 }) { StatusCode = 403 };
    return;
}
```

Where exactly it appears:

Service	Action	Allowed Roles
Auth.API	POST /api/users	Admin
Auth.API	DELETE /api/users/{id}	Admin
ProtocolSite.API	POST / PUT /api/protocols	Admin, CTM
ProtocolSite.API	DELETE /api/protocols	Admin
AdverseEvent.API	POST /api/adverse-events	Admin, CTM, Investigator, DataManager
AdverseEvent.API	PUT /api/adverse-events	Admin, CTM, RegulatoryOfficer, DataManager
DocumentCompliance.API	POST / PUT /api/documents	Admin, CTM, RegulatoryOfficer, DataManager
AnalyticsReport.API	POST /api/kpi-reports	Admin, DataManager
AnalyticsReport.API	PATCH /api/kpi-reports/{id}/review	RegulatoryOfficer
Notification.API	GET /api/notifications (all)	Admin, CTM

All GET actions across all services have **no** `[RoleAuthorize]` — any authenticated user can read.

Filter Execution Order

Every request goes through them in this exact sequence:

```
Request arrives
v
JwtAuthFilter      <- authorization filter, runs first
v (if 401, stop here)
RoleAuthorizeFilter <- authorization filter, runs second
v (if 403, stop here)
ValidateModelFilter <- action filter OnActionExecuting, before action
v (if 422, stop here)
Controller action  <- your actual code runs
v
DomainExceptionFilter <- exception filter, catches DomainException/NotFoundException
v
ActivityLogFilter    <- action filter OnActionExecuted, after action
v
Response sent
```

The whole pipeline is wrapped by `GlobalExceptionHandler` (the outermost try/catch) which catches anything that escapes.

12. Memory Caching — The Complete Concept

What is Cache Memory?

Caching means storing the result of an expensive operation (like a database query) in fast memory so the next identical request gets the answer instantly without hitting the database again.

Without cache, every request runs the full query:

```
Request 1 -> Service -> Repository -> SQL query -> DB -> result (takes 20ms)
Request 2 -> Service -> Repository -> SQL query -> DB -> result (takes 20ms again)
Request 3 -> Service -> Repository -> SQL query -> DB -> result (takes 20ms again)
```

With cache, the first request fills it and the rest are instant:

```
Request 1 -> Service -> cache miss -> SQL query -> DB -> store in cache (20ms)
Request 2 -> Service -> cache hit -> return from memory (0.1ms)
Request 3 -> Service -> cache hit -> return from memory (0.1ms)
```

`IMemoryCache` in .NET stores data **in the application's RAM** — not in a separate server like Redis. It's per-process, meaning each service has its own isolated cache. When the service restarts, the cache is gone.

How it's registered

```
// Every service's Program.cs
builder.Services.AddMemoryCache();
```

`AddMemoryCache` registers `IMemoryCache` as a **singleton** — one instance for the entire application lifetime, shared across all requests and all scoped services. This is correct: the cache must outlive individual requests to be useful.

The problem the project had to solve — wildcard invalidation

`IMemoryCache` stores data under string keys. You can only remove a key if you know its exact name. There is **no built-in "remove all keys matching documents:list:*" operation**.

The list cache keys look like this:

```
documents:list:v0:null:Draft:Protocol:1:50
documents:list:v0:null:null:null:1:50
documents:list:v0:5:Approved:null:2:50
```

When you create or update a document, all of these list keys become stale. But you can't call `_cache.Remove("documents:list:*")` — that doesn't exist.

The solution — version-bump cache invalidation

Every service that uses caching (DocumentService, AdverseEventService, DeviationService, UserService, ProtocolService, SiteService, SiteProtocolService, PatientService, EnrollmentService, VisitService, KpiReportService, NotificationService) implements the same pattern. Here it is using `DocumentService` as the example.

Step 1 — Two types of cache keys

```
private const string VersionKey = "documents:list:version"; // single integer
private const string ItemPrefix = "documents:item";         // per-item keys
```

Item keys — exact, one per document: `documents:item:1`, `documents:item:42`, `documents:item:99`.

List keys — include filter parameters AND a version number:

```
documents:list:v0:null:Draft:Protocol:1:50
documents:list:v1:null:Draft:Protocol:1:50  <- v1 after first mutation
documents:list:v2:null:Draft:Protocol:1:50  <- v2 after second mutation
```

Step 2 — The version counter

```
private int GetVersion() =>
{
    _cache.GetOrCreate(VersionKey, e =>
    {
        e.Priority = CacheItemPriority.NeverRemove; // never expire this key
        return 0; // start at 0
    });
}
```

`GetOrCreate` — if `"documents:list:version"` exists, return its value. If not, create it with value `0` and `NeverRemove` priority. This version integer is the heartbeat of the whole system.

```
private void BumpVersion() =>
{
    _cache.Set(VersionKey, GetVersion() + 1,
        new MemoryCacheEntryOptions { Priority = CacheItemPriority.NeverRemove });
}
```

`BumpVersion` increments the counter by 1. After the first document is created, the version becomes 1; after the second mutation, 2; and so on.

Step 3 — Reading from list cache

```
public async Task<PagedResult<DocumentResponse>> ListAsync(
    long? protocolId, string? status, string? category, int page, int pageSize)
{
    var v = GetVersion(); // read current version, say it's 3
    var key = $"documents:list:v{v}:{protocolId}:{status}:{category}:{page}:{pageSize}";
    // -> "documents:list:v3:null:Draft:Protocol:1:50"

    if (_cache.TryGetValue(key, out PagedResult<DocumentResponse>? cached) && cached is not null)
        return cached; // cache hit, return instantly

    var (items, total) = await _repo.ListAsync(protocolId, status, category, page, pageSize);
    var result = new PagedResult<DocumentResponse> { ... };

    _cache.Set(key, result, CacheDuration); // store under versioned key for 5 minutes
    return result;
}
```

The key embeds the version number `v3`. As long as the version stays at 3, every identical request finds `"documents:list:v3:..."` in cache and returns instantly.

Step 4 — Creating a document

```
public async Task<DocumentResponse> CreateAsync(CreateDocumentRequest req)
{
    var doc = new Document { ... Status = "Draft", ... };
    var created = await _repo.AddAsync(doc); // SQL INSERT, new row in DB

    BumpVersion(); // version goes from 3 -> 4
    return ToResponse(created);
}
```

`BumpVersion()` changes the version from `3` to `4`. That's it. No looping, no hunting for old keys.

Step 5 — Why the old cache entries are now harmless

After `BumpVersion()`, the next list request builds its key with `v4`: `"documents:list:v4:null:Draft:Protocol:1:50"`.

The old `v3` keys still exist in memory but are now **unreachable** — no future request will ever build a `v3` key because `GetVersion()` always returns `4` now. The old entries are **orphaned** — they can never be looked up again. They expire after 5 minutes via `CacheDuration` and are cleaned up by .NET's background expiry process. No manual cleanup needed.

Step 6 — Updating a document (item + list invalidation)

```

public async Task<DocumentResponse?> UpdateAsync(long documentId, UpdateDocumentRequest req)
{
    var doc = await _repo.GetByIdAsync(documentId);
    // ... apply changes ...
    await _repo.UpdateAsync(doc);

    Invalidate(documentId); // both item AND list
    return ToResponse(doc);
}

private void Invalidate(long id)
{
    _cache.Remove($"{ItemPrefix}:{id}"); // removes "documents:item:42" immediately
    BumpVersion();                      // orphans all list keys
}

```

When document 42 is updated: `_cache.Remove("documents:item:42")` removes the specific item cache entry **immediately and precisely** (we know the exact key, so direct removal), and `BumpVersion()` bumps version from 4 to 5, orphaning all list caches silently.

Full flow visualised with real values

```

App starts:
  cache = empty
  version = (not created yet)

GET /api/documents?status=Draft (request 1)
  GetVersion() -> creates "documents:list:version" = 0
  key = "documents:list:v0:null:Draft:null:1:50"
  cache miss -> SQL query -> 12 rows returned
  cache.Set("documents:list:v0:null:Draft:null:1:50", result, 5min)

GET /api/documents?status=Draft (request 2, identical)
  GetVersion() -> returns 0
  key = "documents:list:v0:null:Draft:null:1:50"
  cache HIT -> returns immediately, no SQL

POST /api/documents (someone uploads a new doc)
  INSERT into DB
  BumpVersion() -> "documents:list:version" = 1

GET /api/documents?status=Draft (request 3, after upload)
  GetVersion() -> returns 1
  key = "documents:list:v1:null:Draft:null:1:50" <- different from v0 key!
  cache miss -> SQL query -> 13 rows returned (includes the new doc)
  cache.Set("documents:list:v1:null:Draft:null:1:50", result, 5min)

Old "v0" key still in RAM, expires in ~5 minutes, never looked up again.

```

The two cache strategies side by side

	Item cache	List cache
Key format	<code>documents:item:42</code>	<code>documents:list:v3:null:Draft:null:1:50</code>
Invalidation	<code>_cache.Remove(exactKey)</code>	<code>BumpVersion()</code> — orphan by version change
Why different?	Exact ID known -> direct removal	Many filter combinations -> can't enumerate all keys
After invalidation	Entry gone immediately	Entry still in RAM but unreachable, expires in 5 min

Why NeverRemove on the version key?

```
e.Priority = CacheItemPriority.NeverRemove;
```

When the server runs low on memory, .NET's `IMemoryCache` starts evicting cache entries, low-priority ones first. If the version key got evicted, `GetVersion()` would re-create it at `0` — and suddenly all the `v3`, `v4`, `v5` keys built so far would never expire naturally (they'd always be "future" versions that nobody looks up, but they'd also never be regenerated correctly). Setting `NeverRemove` means the version counter survives memory pressure events, keeping the whole system coherent.

Cache duration — why 5 minutes?

```
private static readonly TimeSpan CacheDuration = TimeSpan.FromMinutes(5);
```

5 minutes is a balance. Long enough to meaningfully reduce DB load on repeated reads. Short enough that even if cache invalidation somehow failed, stale data is at most 5 minutes old. For clinical trial data this is acceptable — protocol statuses and document lists don't change every second.

13. Complete Learning Summary

1. Microservices Architecture — The Big Picture

A microservices solution is not one big application but a collection of small independent services, each with its own process, port, and responsibility. LifeTrack satisfies the core requirement with **8 independent services** (Gateway 5046, Authentication 5033, ProtocolSite 5193, Patient 5095, AdverseEvent 5159, DocumentCompliance 5232, AnalyticsReport 5062, Notification 5210), **2 databases** (`governanceDb` owned by Auth + DocumentCompliance + Notification; `clinicalDb` shared by ProtocolSite + Patient + AdverseEvent + AnalyticsReport), and **1 shared class library** (`Shared.CL`) providing cross-cutting concerns without code duplication.

2. JWT — From Scratch to End

Issuing (only Authentication.API): user submits email + password → `AuthService` fetches the user → `BCryptPasswordHasher.Verify()` checks the password → `JwtTokenService.Generate()` builds claims (`sub`, `email`, `jti`, `NameIdentifier`, `Name`, `Role`) → `SymmetricSecurityKey` + `HmacSha256` signs the token → `WriteToken()` produces the three-part string → Angular stores it.

Validating (every downstream service independently): `JwtAuthFilter` reads the `Authorization: Bearer` header → reads `Jwt:Secret` from its own `appsettings.json` (same secret as Auth.API) → `ValidateToken()` verifies signature, expiry, issuer, audience in one call → sets `HttpContext.User`. Services never call Auth.API to validate — the shared secret is enough. This is the stateless nature of JWT.

Special saga — patient registration: create User → auto-login for JWT → call Patient.API with that JWT → compensating `TryDeleteUserAsync` on failure.

3. Gateway.API — Ocelot Routing

The Gateway is deliberately minimal — a reverse proxy with zero business logic. `ocelot.json` maps every `/gateway/<resource>` path to a downstream port. Every resource has two entries (exact path + wildcard). Ocelot passes the `Authorization` header through unchanged and never validates JWT. `reloadOnChange: true` allows route changes without restart. No `MapControllers()` — zero controllers.

4. Shared.CL — The Security and Error Brain

A class library (not a web project) using `FrameworkReference`. **Middleware:** `GlobalExceptionHandler` (outermost try/catch, hides details for 500s, adds TraceId). **Filters:** `JwtAuthFilter`, `RoleAuthorizeFilter` + `RoleAuthorizeAttribute`, `ValidateModelFilter` (422), `DomainExceptionHandler` (400/404), `ActivityLogFilter`. **DTOs:** `ApiResponse<T>`, `PagedResponse<T>`, `ErrorDetail`. **Exceptions:** `DomainException`, `NotFoundException`. **Enum:** `RolesEnum` (Admin=1 through DataManager=6, matching the seeded Roles table exactly).

5. Authentication.API — Every Layer

Models (`User`, `Role`, `AuditLog`, `AuditEntry`) with data annotations creating SQL constraints. `AuthDbContext` auto-audit `SaveChangesAsync` override using `ChangeTracker`. `DbSeeder` idempotent seeding using `AnyAsync()` guards with `BCrypt` `workFactor: 12`. `BCryptPasswordHasher` (`Hash/Verify` with `SaltParseException` catch). `JwtTokenService` builds claims and signs with `HmacSha256`. `AuthService` normalizes email to lowercase (prevents enumeration), combines null-check and password-verify in one `if`, compensating saga. `UserService` version-bump cache invalidation. Controllers with `[AllowAnonymous]` on login, `GetUserId()` helper, guard clauses protecting UserID 1.

6. ProtocolSite.API — Business Rules and Schema Design

`Protocol`, `Site`, `SiteProtocol` (junction table), `InvestigatorID` as cross-service reference with no EF FK, `Phase` moved from Protocol to SiteProtocol via migration. `OnDelete(Restrict)` preventing cascade deletes, `ValueGeneratedNever()` on RoleID. Business rules: EndDate ≥ StartDate, cannot terminate with active assignments, cannot close site with assignments, cannot delete Active entities, five-step validation chain. Cache version pattern introduced here.

7. AdverseEvent.API — Cross-Service Design

`AdverseEventRecord` named differently from table to avoid namespace collision. All IDs cross-service (raw `long`, no FK). `ExcludeFromMigrations()` — table used but not owned. Status hardcoded `"Reported"` on create. `"Closed"` status blocked. Role split: Investigator can report (POST) but not update (PUT); RegulatoryOfficer can update but not create. `_db.Update()` explicit.

8. DocumentCompliance.API — Schema Drift and Governance

Uses `governanceDb` not `clinicalDb` (governance data). `PendingModelChangesWarning` suppressed due to schema drift (Type→Category rename, ProtocolID nullability change). Raw SQL migration `AddFileColumns` with `IF NOT EXISTS` guards. `DocumentVersion.cs` planned but unimplemented (no DbSet/migration/service). `Approved → Draft` business rule.

9. Patient.API — Cross-Entity Status & Relationships

`Patient → Enrollment → Visit` chain. Two delete behaviors: Patient→Enrollment (Restrict), Enrollment→Visit (Cascade). Unique filtered index on UserID (`[UserID] IS NOT NULL`). `ExecuteUpdateAsync` bulk update bypassing the change tracker. `SyncPatientStatusAsync` derives patient status from all enrollments (Active > Screening > Completed > Withdrawn). Withdrawal terminal rule. IDOR-safe patient create with `req with { UserID = callerId }`.

10. AnalyticsReport.API — Reporting Snapshot

Fat flat `KpiReport` entity (rates as double, counts as int). Explicit `CurrentUserID` property pattern instead of `IHttpContextAccessor`. Four incremental migrations. `ReviewAsync` sets `Status = "Reviewed"` via `[HttpPatch]` sub-resource route. Create = Admin/DataManager; review = RegulatoryOfficer only.

11. Notification.API — Row-Level Security

Composite index `(UserID, Status)` for the bell-count query. IDOR protection: `MarkReadAsync / DeleteAsync` check `n.UserID == callerUserId`, throw `UnauthorizedAccessException` → 403 Forbid. Three access patterns: GET /my (self-filtered), GET / (Admin/CTM), POST (all six roles). Uses `governanceDb`.

12. EF Core Patterns — Used Across All Services

Pattern	What it does	Where used
<code>AddScoped<DbContext></code>	One DbContext per HTTP request	Every service
<code>AsQueryable()</code> + chained <code>.Where()</code>	One SQL query with optional filters	All repositories
<code>CountAsync()</code> before <code>Skip/Take</code>	Total count for pagination	All list repos
<code>AsNoTracking()</code>	Skips change tracker for reads	ProtocolSite, Patient
<code>Include()</code> eager loading	Single SQL JOIN, no N+1	UserRepository
<code>ExcludeFromMigrations()</code>	Use table without owning schema	AE, DocCompliance, Patient, Analytics, Notification
<code>ValueGeneratedNever()</code>	Disable IDENTITY on explicit-ID tables	Roles in Auth.API
<code>OnDelete(Restrict/Cascade)</code>	Block or cascade deletes at DB level	SiteProtocol, Patient
<code>ExecuteUpdateAsync</code>	Bulk UPDATE, bypass change tracker	PatientRepository
<code>SaveChangesAsync</code> override	Auto-audit every entity change	Every clinical DbContext

13. Caching — The Version-Bump Pattern

Problem: `IMemoryCache` has no wildcard removal. **Solution:** embed a version number in every list cache key; to invalidate all list keys, increment the version, orphaning old keys which expire naturally. **Two-track:** item keys removed directly (`_cache.Remove`), list keys orphaned (`BumpVersion`). `NeverRemove` priority on version key survives memory pressure. 5-minute `CacheDuration` balances load reduction against staleness.

What This Whole Project Demonstrates

By building LifeTrack you have hands-on experience with: designing and implementing a real microservices architecture with service boundaries; JWT authentication from token issuance through cross-service validation; the API gateway pattern with

Ocelot; shared cross-cutting concerns via a class library; EF Core code-first with migrations, relationships, and auto-auditing; filter pipeline design in ASP.NET Core; memory caching with a custom invalidation strategy; domain-driven business rule enforcement via typed exceptions; cross-service data references without distributed transactions; real migration management including schema drift and raw SQL migrations; and role-based access control with typed enums. That is a genuinely production-relevant backend architecture — not a tutorial project.